

CLARITY AI EXECUTIVE DASHBOARD

Full project handoff: architecture, API contract, subscription tiers and complete source code for the FastAPI backend and native Kotlin / Jetpack Compose Android app.

Generated October 06, 2026

1. Use this with Claude Code

Best way: unzip clarity-ai.zip and let Claude Code read the folder directly. Copy-pasting code out of a PDF can mangle indentation, so the folder is the source of truth and this PDF is the readable reference.

```
unzip clarity-ai.zip && cd clarity-ai
claude # start Claude Code in the project root
```

Starter prompt (paste into Claude Code)

Read README.md, backend/ and android/. This is a FastAPI + Kotlin/Compose app that pulls Microsoft Clarity telemetry, sanitises it, streams a Claude-written engineering backlog as NDJSON, and exports an executive PDF. It has Free/Pro/Max subscription tiers enforced server-side.

Tasks:

- Backend: create a venv, install requirements.txt, run `uvicorn main:app`, and fix any errors. Write pytest tests for normalize(), compute_kpis(), top_targets(), sanitize_path(), SectionGate and the tier/quota enforcement in main.py (use httpx ASGITransport, mock Clarity and Anthropic).
- Android: open android/ as a Gradle project, run ./gradlew :app:assembleDebug (generate the wrapper first with `gradle wrapper` if missing) and fix compile errors. Verify the markdown renderer dependency/API in DashboardScreen.kt against the latest release.
- Add an X-Dev-Tier header to ApiClient.kt for debug builds only (BuildConfig.DEBUG) so tiers can be tested without Google Play.
- Do not change the tier limits, the Claude prompt template text, or the NDJSON event contract without asking me first.

If you only have this PDF

- Recreate each file at the exact path in its header in section 8, character for character.
- Lines ending in the ↵ symbol were wrapped for printing: join them with the next line (drop the 4-space continuation indent).
- Emoji are printed as <U+1F6A8> style code points because the PDF font has no emoji glyphs. Restore them as real characters: U+1F6A8 = siren, U+1F4C9 = chart decreasing, U+1F680 = rocket. They appear in the Claude prompt template and the ROADMAP_MARKER in clarity_ai.py, and the marker must match the prompt header exactly.

2. Architecture

Two environments. The Android app never talks to Clarity or Anthropic directly; all keys live on the backend.

Layer	Responsibility
Android (Kotlin, Compose, Material 3)	Dashboard UI, KPI cards, timeframe dropdown, shimmer loaders, streamed markdown card, paywall (Play Billing), on-device PDF generation (PdfDocument) and Android share sheet via FileProvider.

Layer	Responsibility
FastAPI backend	Auth by device ID, tier and quota enforcement, Clarity fetch with caching and retries, sanitisation, KPI/health score engine, Claude streaming, Play subscription verification, daily snapshot job.
SQLite (aiosqlite)	users/tiers, monthly usage, tracked projects, daily Clarity snapshots, API response cache.
Microsoft Clarity	Data Export API: GET /export-data/api/v1/project-live-insights (Bearer token). Last 1-3 days only, 10 requests/project/day.
Anthropic API	Streaming messages call. Model from CLAUDE_MODEL (default claude-sonnet-5-5).

Request flow for POST /api/v1/analytics/audit

- Validate device ID, load user and tier; check timeframe allowed for tier (403 UPGRADE_REQUIRED).
- Register project against tier project limit (403 PROJECT_LIMIT); atomically reserve one monthly audit (429 QUOTA_EXCEEDED).
- Fetch Clarity data by URL dimension (live for 1-3 days, snapshots for longer ranges); Max also fetches Device dimension. Refund the reserved audit on any failure.
- Normalise using a whitelist of numeric fields; sanitise page paths (host, query, fragment dropped; ID-like segments become :id); scrub emails/IPs from the final JSON.
- Compute KPIs and session health score, rank top friction targets (3 for Free, 10 otherwise).
- Stream NDJSON events: meta, delta..., [locked], done (or error). Free tier output is cut before the roadmap header.

3. Subscription tiers

Defined once in backend/services/tiers.py and mirrored for display in PaywallSheet.kt. The backend is authoritative; the app only uses the limits to show locks and the paywall.

Feature	Free	Pro	Max
Audits per month	3	60	600 (fair use)
Timeframes	Today, Last 3 days	+ Last 7 days	+ Last 30 days, Custom (up to 90 days)
Top friction targets	3	10	10
Report sections	Hotfixes + Trends	+ Strategic Roadmap	+ Strategic Roadmap
PDF export and share	No	Yes	Yes, white-label
Device breakdown	No	No	Yes
Projects	1	5	25
Play product ID	-	clarity_pro	clarity_max

Purchases are verified server-side with the Google Play Developer API (subscriptionsv2). Expired subscriptions automatically fall back to Free.

4. API contract

Endpoint	Purpose
GET /health	Liveness.
GET /api/v1/account/entitlements	Tier, expiry, monthly usage and feature flags for the calling device.
POST /api/v1/subscription/verify	Body {purchase_token, product_id}. Verifies with Google Play, upgrades the tier, returns entitlements.

Endpoint	Purpose
POST /api/v1/analytics/audit	Body {project_id, timeframe: TODAY LAST_3_DAYS LAST_WEEK LAST_MONTH CUSTOM, start_date?, end_date?}. Streams application/x-ndjson.

All requests require header X-Device-Id (16-64 chars, alphanumeric or dash). Errors use {"detail": {"code", "message", "required_tier"}}.

NDJSON events

```
{
  "type": "meta",
  "tier": "PRO",
  "kpis": {...},
  "notes": [...],
  "days_covered": 3,
  "usage": {
    "used": 4,
    "limit": 60,
    "period": "2026-10"
  }
}
{"type": "delta", "text": "### <U+1F6A8> Immediate Hotfixes ..."}
{"type": "locked", "section": "roadmap", "message": "Upgrade to Pro to unlock ..."} // Free tier only
{"type": "done"}
{"type": "error", "code": "AI_RATE_LIMIT", "message": "..."}

```

Error codes

Code	HTTP	Meaning
UPGRADE_REQUIRED	403	Timeframe not included in the plan (required_tier set).
PROJECT_LIMIT / QUOTA_EXCEEDED	403 / 429	Plan project or monthly audit limit reached.
CLARITY_AUTH / CLARITY_RATE_LIMIT / NO_DATA	502 / 429 / 404	Clarity rejected token, quota exhausted, or no data returned.
AI_RATE_LIMIT / AI_ERROR / AI_EMPTY	429 / 502	Claude errors; the audit is refunded.
INVALID_PURCHASE / SUBSCRIPTION_INACTIVE	402	Play verification failed.

5. Known constraints and decisions

- **Endpoint:** the brief's /v1/projects/{id}/metrics does not exist; the real Clarity export endpoint is project-live-insights.
- **History:** Clarity returns at most 3 days, so week/month/custom ranges come from daily snapshots the server records; coverage is reported in meta.notes.
- **Targets:** Clarity exports metrics by dimension (URL, Device...) not CSS selectors/View IDs, so friction targets are page paths.
- **Identity:** accounts are keyed by an install-generated device ID, so a reinstall resets the free quota. Add real auth (Firebase Auth / Play Integrity) before launch.
- **Dev override:** DEV_ALLOW_TIER_OVERRIDE must stay false in production.
- **Markdown lib:** verify com.mikepenz multiplatform-markdown-renderer-m3 version and the markdownColor parameters when syncing.

6. Environment

Variable	Purpose
ANTHROPIC_API_KEY, CLAUDE_MODEL	Claude access and model name.
CLARITY_API_TOKEN / CLARITY_TOKENS_JSON	Single token, or per-project map {"projectId": "token"} (map wins).
DATABASE_PATH, CACHE_TTL_SECONDS, SNAPSHOT_HOUR_UTC	SQLite file, Clarity cache TTL (default 3600s), hour after which the daily snapshot job runs.
PLAY_PACKAGE_NAME, GOOGLE_SERVICE_ACCOUNT_FILE	Play subscription verification.
DEV_ALLOW_TIER_OVERRIDE	Testing only: accepts X-Dev-Tier header.

7. File tree

```
clarity-ai/  
  backend/requirements.txt  
  backend/.env.example  
  backend/config.py  
  backend/models.py  
  backend/services/tiers.py  
  backend/services/storage.py  
  backend/services/billing.py  
  backend/services/clarity_ai.py  
  backend/main.py  
  android/settings.gradle.kts  
  android/build.gradle.kts  
  android/gradle.properties  
  android/gradle/wrapper/gradle-wrapper.properties  
  android/.gitignore  
  android/app/build.gradle.kts  
  android/app/proguard-rules.pro  
  android/app/src/main/AndroidManifest.xml  
  android/app/src/main/res/xml/file_paths.xml  
  android/app/src/main/res/values/themes.xml  
  android/app/src/main/java/com/example/clarityai/MainActivity.kt  
  android/app/src/main/java/com/example/clarityai/data/ApiClient.kt  
  android/app/src/main/java/com/example/clarityai/billing/BillingManager.kt  
  android/app/src/main/java/com/example/clarityai/pdf/PdfExportUtility.kt  
  android/app/src/main/java/com/example/clarityai/ui/Theme.kt  
  android/app/src/main/java/com/example/clarityai/ui/DashboardViewModel.kt  
  android/app/src/main/java/com/example/clarityai/ui/DashboardScreen.kt  
  android/app/src/main/java/com/example/clarityai/ui/PaywallSheet.kt  
  README.md
```

8a. Source: backend

backend/requirements.txt

9 lines

```
fastapi>=0.115
uvicorn[standard]>=0.30
httpx>=0.27
anthropic>=0.40
pydantic>=2.7
pydantic-settings>=2.3
aiosqlite>=0.20
google-auth>=2.30
requests>=2.32
```

backend/.env.example

17 lines

```
ANTHROPIC_API_KEY=sk-ant-...
CLAUDE_MODEL=claude-sonnet-5-5

# Clarity: one token for all projects, or a per-project map (map wins)
CLARITY_API_TOKEN=
CLARITY_TOKENS_JSON={"yourProjectId":"yourClarityApiToken"}

DATABASE_PATH=clarity_ai.db
CACHE_TTL_SECONDS=3600
SNAPSHOT_HOUR_UTC=23

# Google Play subscription verification
PLAY_PACKAGE_NAME=com.example.clarityai
GOOGLE_SERVICE_ACCOUNT_FILE=/secrets/play-service-account.json

# NEVER enable in production: lets a client self-assign a tier via X-Dev-Tier
DEV_ALLOW_TIER_OVERRIDE=false
```

backend/config.py

38 lines

```
from __future__ import annotations

import json
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    anthropic_api_key: str
    claude_model: str = "claude-sonnet-5-5"

    clarity_api_token: str = ""
    clarity_tokens_json: str = "{}"
    clarity_base_url: str = "https://www.clarity.ms/export-data/api/v1"

    database_path: str = "clarity_ai.db"
    cache_ttl_seconds: int = 3600
    snapshot_hour_utc: int = 23

    play_package_name: str = ""
    google_service_account_file: str = ""

    dev_allow_tier_override: bool = False

    def token_for(self, project_id: str) -> str | None:
        try:
            mapping = json.loads(self.clarity_tokens_json or "{}")
        except json.JSONDecodeError:
            mapping = {}
        return mapping.get(project_id) or self.clarity_api_token or None

@lru_cache
def get_settings() -> Settings:
    return Settings() # type: ignore[call-arg]
```

backend/models.py

48 lines

```
from __future__ import annotations

from datetime import date, datetime, timezone
```

```

from enum import Enum

from pydantic import BaseModel, Field, model_validator

class Timeframe(str, Enum):
    TODAY = "TODAY"
    LAST_3_DAYS = "LAST_3_DAYS"
    LAST_WEEK = "LAST_WEEK"
    LAST_MONTH = "LAST_MONTH"
    CUSTOM = "CUSTOM"

class Tier(str, Enum):
    FREE = "FREE"
    PRO = "PRO"
    MAX = "MAX"

class AuditRequest(BaseModel):
    project_id: str = Field(min_length=1, max_length=128, pattern=r"^[A-Za-z0-9_\-]+$")
    timeframe: Timeframe
    start_date: date | None = None
    end_date: date | None = None

    @model_validator(mode="after")
    def _validate_dates(self) -> "AuditRequest":
        if self.timeframe is Timeframe.CUSTOM:
            if not self.start_date or not self.end_date:
                raise ValueError("start_date and end_date are required for CUSTOM timeframe")
            if self.start_date > self.end_date:
                raise ValueError("start_date must be on or before end_date")
            if self.end_date > datetime.now(timezone.utc).date():
                raise ValueError("end_date cannot be in the future")
            if (self.end_date - self.start_date).days + 1 > 90:
                raise ValueError("CUSTOM range cannot exceed 90 days")
        else:
            self.start_date = None
            self.end_date = None
        return self

class SubscriptionVerifyRequest(BaseModel):
    purchase_token: str = Field(min_length=10, max_length=4096)
    product_id: str = Field(min_length=1, max_length=128)

```

backend/services/tiers.py

69 lines

```

from __future__ import annotations

from dataclasses import dataclass

from models import Tier, Timeframe

@dataclass(frozen=True)
class TierPolicy:
    tier: Tier
    monthly_audits: int
    allowed_timeframes: frozenset[Timeframe]
    max_custom_days: int
    top_elements: int
    full_roadmap: bool
    pdf_export: bool
    white_label_pdf: bool
    device_breakdown: bool
    max_projects: int

    def to_features(self) -> dict:
        return {
            "allowed_timeframes": sorted(t.value for t in self.allowed_timeframes),
            "pdf_export": self.pdf_export,
            "white_label_pdf": self.white_label_pdf,
            "top_elements": self.top_elements,
            "device_breakdown": self.device_breakdown,
            "full_roadmap": self.full_roadmap,
            "max_projects": self.max_projects,
        }

T = Timeframe
POLICIES: dict[Tier, TierPolicy] = {
    Tier.FREE: TierPolicy(
        tier=Tier.FREE, monthly_audits=3,
        allowed_timeframes=frozenset({T.TODAY, T.LAST_3_DAYS}),
    ),

```

```

        max_custom_days=0, top_elements=3, full_roadmap=False, pdf_export=False,
        white_label_pdf=False, device_breakdown=False, max_projects=1,
    ),
    Tier.PRO: TierPolicy(
        tier=Tier.PRO, monthly_audits=60,
        allowed_timeframes=frozenset({T.TODAY, T.LAST_3_DAYS, T.LAST_WEEK}),
        max_custom_days=0, top_elements=10, full_roadmap=True, pdf_export=True,
        white_label_pdf=False, device_breakdown=False, max_projects=5,
    ),
    Tier.MAX: TierPolicy(
        tier=Tier.MAX, monthly_audits=600, # fair-use ceiling
        allowed_timeframes=frozenset(Timeframe),
        max_custom_days=90, top_elements=10, full_roadmap=True, pdf_export=True,
        white_label_pdf=True, device_breakdown=True, max_projects=25,
    ),
}

TIMEFRAME_MIN_TIER: dict[Timeframe, Tier] = {
    T.TODAY: Tier.FREE, T.LAST_3_DAYS: Tier.FREE, T.LAST_WEEK: Tier.PRO,
    T.LAST_MONTH: Tier.MAX, T.CUSTOM: Tier.MAX,
}

_ORDER = [Tier.FREE, Tier.PRO, Tier.MAX]

def required_tier(tf: Timeframe) -> Tier:
    return TIMEFRAME_MIN_TIER[tf]

def next_tier(t: Tier) -> Tier:
    i = _ORDER.index(t)
    return _ORDER[min(i + 1, len(_ORDER) - 1)]

```

backend/services/storage.py

177 lines

```

from __future__ import annotations

import asyncio
import json
import time
from dataclasses import dataclass
from datetime import date, datetime, timezone
from typing import Any

import aiosqlite

from models import Tier

SCHEMA = """
CREATE TABLE IF NOT EXISTS users(
    device_id TEXT PRIMARY KEY, tier TEXT NOT NULL DEFAULT 'FREE',
    expires_at TEXT, purchase_token TEXT, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS usage(
    device_id TEXT NOT NULL, period TEXT NOT NULL, count INTEGER NOT NULL DEFAULT 0,
    PRIMARY KEY(device_id, period));
CREATE TABLE IF NOT EXISTS device_projects(
    device_id TEXT NOT NULL, project_id TEXT NOT NULL, PRIMARY KEY(device_id, project_id));
CREATE TABLE IF NOT EXISTS snapshots(
    project_id TEXT NOT NULL, day TEXT NOT NULL, payload TEXT NOT NULL,
    PRIMARY KEY(project_id, day));
CREATE TABLE IF NOT EXISTS api_cache(
    key TEXT PRIMARY KEY, payload TEXT NOT NULL, fetched_at REAL NOT NULL);
"""

@dataclass
class UserRecord:
    device_id: str
    tier: Tier
    expires_at: datetime | None

class Storage:
    def __init__(self, path: str) -> None:
        self._path = path
        self._db: aiosqlite.Connection | None = None
        self._lock = asyncio.Lock()

    async def init(self) -> None:
        self._db = await aiosqlite.connect(self._path)
        self._db.row_factory = aiosqlite.Row
        await self._db.executescript(SCHEMA)
        await self._db.commit()

    async def close(self) -> None:

```

```

    if self._db:
        await self._db.close()

@property
def db(self) -> aiosqlite.Connection:
    assert self._db is not None, "Storage not initialised"
    return self._db

# ---- users / tiers -----
async def get_user(self, device_id: str) -> UserRecord:
    async with self._lock:
        await self.db.execute(
            "INSERT OR IGNORE INTO users(device_id, tier, created_at) VALUES(?, 'FREE', ?)",
            (device_id, datetime.now(timezone.utc).isoformat()),
        )
        cur = await self.db.execute("SELECT tier, expires_at FROM users WHERE device_id=?", (device_id,))
        row = await cur.fetchone()
        tier = Tier(row["tier"])
        expires = datetime.fromisoformat(row["expires_at"]) if row["expires_at"] else None
        if tier is not Tier.FREE and expires and expires < datetime.now(timezone.utc):
            await self.db.execute("UPDATE users SET tier='FREE' WHERE device_id=?", (device_id,))
            tier = Tier.FREE
        await self.db.commit()
        return UserRecord(device_id, tier, expires)

async def set_tier(self, device_id: str, tier: Tier, expires_at: datetime, token: str) -> None:
    async with self._lock:
        # A purchase token belongs to one device at a time (latest verification wins).
        await self.db.execute(
            "UPDATE users SET tier='FREE', purchase_token=NULL WHERE purchase_token=? AND device_id!=?",
            (token, device_id),
        )
        await self.db.execute(
            "INSERT INTO users(device_id, tier, expires_at, purchase_token, created_at) VALUES(?,?,?,?,?) "
            "ON CONFLICT(device_id) DO UPDATE SET tier=excluded.tier, expires_at=excluded.expires_at, "
            "purchase_token=excluded.purchase_token",
            (device_id, tier.value, expires_at.isoformat(), token, datetime.now(timezone.utc).isoformat()),
        )
        await self.db.commit()

# ---- quota -----
async def usage(self, device_id: str, period: str) -> int:
    cur = await self.db.execute("SELECT count FROM usage WHERE device_id=? AND period=?", (device_id, period))
    row = await cur.fetchone()
    return int(row["count"]) if row else 0

async def try_consume(self, device_id: str, period: str, limit: int) -> int | None:
    """Atomically reserve one audit. Returns the new count, or None if the quota is exhausted."""
    async with self._lock:
        used = await self.usage(device_id, period)
        if used >= limit:
            return None
        await self.db.execute(
            "INSERT INTO usage(device_id, period, count) VALUES(?,?,1) "
            "ON CONFLICT(device_id, period) DO UPDATE SET count=count+1",
            (device_id, period),
        )
        await self.db.commit()
        return used + 1

async def refund(self, device_id: str, period: str) -> None:
    async with self._lock:
        await self.db.execute(
            "UPDATE usage SET count=MAX(count-1,0) WHERE device_id=? AND period=?", (device_id, period)
        )
        await self.db.commit()

# ---- projects -----
async def register_project(self, device_id: str, project_id: str, max_projects: int) -> bool:
    async with self._lock:
        cur = await self.db.execute(
            "SELECT 1 FROM device_projects WHERE device_id=? AND project_id=?", (device_id, project_id)
        )
        if await cur.fetchone():
            return True
        cur = await self.db.execute("SELECT COUNT(*) AS n FROM device_projects WHERE device_id=?", (device_id,))
        if (await cur.fetchone())["n"] >= max_projects:
            return False
        await self.db.execute(
            "INSERT INTO device_projects(device_id, project_id) VALUES(?,?)", (device_id, project_id)
        )
        await self.db.commit()
        return True

async def tracked_projects(self) -> list[str]:
    cur = await self.db.execute("SELECT DISTINCT project_id FROM device_projects")

```

```

        return [r["project_id"] for r in await cur.fetchall()]

# ---- snapshots -----
async def save_snapshot(self, project_id: str, day: date, normalized: dict[str, Any]) -> None:
    async with self._lock:
        await self.db.execute(
            "INSERT INTO snapshots(project_id, day, payload) VALUES(?,?,?) "
            "ON CONFLICT(project_id, day) DO UPDATE SET payload=excluded.payload",
            (project_id, day.isoformat(), json.dumps(normalized)),
        )
        await self.db.commit()

async def has_snapshot(self, project_id: str, day: date) -> bool:
    cur = await self.db.execute(
        "SELECT 1 FROM snapshots WHERE project_id=? AND day=?", (project_id, day.isoformat())
    )
    return await cur.fetchone() is not None

async def get_snapshots(self, project_id: str, start: date, end: date) -> list[tuple[date, dict[str, Any]]]:
    cur = await self.db.execute(
        "SELECT day, payload FROM snapshots WHERE project_id=? AND day BETWEEN ? AND ? ORDER BY day",
        (project_id, start.isoformat(), end.isoformat()),
    )
    return [(date.fromisoformat(r["day"]), json.loads(r["payload"])) for r in await cur.fetchall()]

# ---- response cache (protects Clarity's 10 requests/project/day cap) -
async def cache_get(self, key: str, ttl: int) -> Any | None:
    cur = await self.db.execute("SELECT payload, fetched_at FROM api_cache WHERE key=?", (key,))
    row = await cur.fetchone()
    if row and time.time() - row["fetched_at"] < ttl:
        return json.loads(row["payload"])
    return None

async def cache_put(self, key: str, payload: Any) -> None:
    async with self._lock:
        await self.db.execute(
            "INSERT INTO api_cache(key, payload, fetched_at) VALUES(?,?,?) "
            "ON CONFLICT(key) DO UPDATE SET payload=excluded.payload, fetched_at=excluded.fetched_at",
            (key, json.dumps(payload), time.time()),
        )
        await self.db.commit()

```

backend/services/billing.py

75 lines

```

from __future__ import annotations

import asyncio
import re
from datetime import datetime, timezone

import httpx
from google.auth.transport.requests import Request as GoogleRequest
from google.oauth2 import service_account

from config import Settings
from models import Tier

# Play Console subscription product IDs -> tier
PRODUCT_TIERS: dict[str, Tier] = {"clarity_pro": Tier.PRO, "clarity_max": Tier.MAX}
ACTIVE_STATES = {
    "SUBSCRIPTION_STATE_ACTIVE",
    "SUBSCRIPTION_STATE_IN_GRACE_PERIOD",
    "SUBSCRIPTION_STATE_CANCELED", # canceled but paid through expiry
}
API = "https://androidpublisher.googleapis.com/androidpublisher/v3/applications"

class BillingError(Exception):
    def __init__(self, status: int, code: str, message: str) -> None:
        super().__init__(message)
        self.status, self.code, self.message = status, code, message

def _parse_ts(value: str) -> datetime:
    value = re.sub(r"(\.\d{6})\d+", r"\1", value.replace("Z", "+00:00"))
    return datetime.fromisoformat(value).astimezone(timezone.utc)

class PlayBillingVerifier:
    def __init__(self, settings: Settings, http: httpx.AsyncClient) -> None:
        self._s, self._http = settings, http
        self._creds: service_account.Credentials | None = None

    async def _access_token(self) -> str:
        if not self._s.google_service_account_file or not self._s.play_package_name:

```

```

        raise BillingError(503, "BILLING_NOT_CONFIGURED", "Play billing verification is not configured.")
    if self._creds is None:
        self._creds = service_account.Credentials.from_service_account_file(
            self._s.google_service_account_file,
            scopes=["https://www.googleapis.com/auth/androidpublisher"],
        )
    if not self._creds.valid:
        await asyncio.to_thread(self._creds.refresh, GoogleRequest())
    return self._creds.token

async def verify(self, purchase_token: str, product_id: str) -> tuple[Tier, datetime]:
    tier = PRODUCT_TIERS.get(product_id)
    if tier is None:
        raise BillingError(400, "UNKNOWN_PRODUCT", f"Unknown product '{product_id}'.")
    token = await self._access_token()
    url = f"{API}/{self._s.play_package_name}/purchases/subscriptionsv2/tokens/{purchase_token}"
    try:
        r = await self._http.get(url, headers={"Authorization": f"Bearer {token}"})
    except httpx.HTTPError as exc:
        raise BillingError(502, "PLAY_UNREACHABLE", "Could not reach Google Play to verify the purchase.") from exc
    if r.status_code in (400, 404, 410):
        raise BillingError(402, "INVALID_PURCHASE", "Google Play does not recognise this purchase.")
    if r.status_code != 200:
        raise BillingError(502, "PLAY_ERROR", f"Google Play returned HTTP {r.status_code}.")
    data = r.json()
    if data.get("subscriptionState") not in ACTIVE_STATES:
        raise BillingError(402, "SUBSCRIPTION_INACTIVE", "This subscription is not active.")
    for item in data.get("lineItems", []):
        if item.get("productId") == product_id:
            expiry = _parse_ts(item["expiryTime"])
            if expiry <= datetime.now(timezone.utc):
                raise BillingError(402, "SUBSCRIPTION_EXPIRED", "This subscription has expired.")
            return tier, expiry
    raise BillingError(402, "PRODUCT_MISMATCH", "Purchase does not match the requested product.")

```

backend/services/clarity_ai.py

448 lines

```

"""Clarity data fetcher, sanitizer, KPI engine and Claude invocation."""
from __future__ import annotations

import asyncio
import json
import logging
import re
from contextlib import aclosing
from dataclasses import dataclass, field
from datetime import date, datetime, timedelta, timezone
from typing import Any, AsyncIterator
from urllib.parse import urlsplit

import anthropic
import httpx

from config import Settings
from models import AuditRequest, Timeframe
from services.storage import Storage
from services.tiers import TierPolicy, required_tier

log = logging.getLogger("clarity_ai")

PROMPT_TEMPLATE = """You are a Staff Product Manager and Lead UX Researcher. Analyze this sanitized Microsoft Clarity JSON telemetry payload and generate a strict, non-conversational engineering backlog:
[INSERT SANITIZED CLARITY JSON HERE]

Format the output strictly in Markdown with these specific headers:
### <U+1F6A8> Immediate Hotfixes (High Frustration)
- [Bullet points containing: Specific element, inferred technical cause, concrete fix action]

### <U+1F4C9> Friction Trends & User Drop-off
- [Bullet points detailing behavioral friction patterns detected across screens]

### <U+1F680> Strategic Next-Sprint Product Roadmap
- [Prioritized features derived directly from conversion drop-offs to improve retention]
"""
SYSTEM_PROMPT = "Respond only with the requested Markdown report. No preamble, no closing remarks."
ROADMAP_MARKER = "### <U+1F680>"

class AuditError(Exception):
    def __init__(self, status: int, code: str, message: str, required_tier: str | None = None) -> None:
        super().__init__(message)
        self.status, self.code, self.message, self.required_tier = status, code, message, required_tier

def utc_today() -> date:

```

```

    return datetime.now(timezone.utc).date()

# -----
# Normalisation helpers
# -----
TRACKED_METRICS = {
    "Traffic", "DeadClickCount", "RageClickCount", "QuickbackClick",
    "ExcessiveScroll", "ScriptErrorCount", "ErrorClickCount",
}

def _int(v: Any) -> int:
    try:
        return int(float(v))
    except (TypeError, ValueError):
        return 0

def _dim_value(info: dict[str, Any], dimension: str) -> str:
    target = dimension.lower()
    for k, v in info.items():
        if k.lower() == target and v not in (None, ""):
            return str(v)
    return "(all)"

def _add(rows: dict[str, dict[str, int]], dim: str, vals: dict[str, int]) -> None:
    dest = rows.setdefault(dim, {})
    for k, v in vals.items():
        dest[k] = dest.get(k, 0) + v

def normalize(raw: list[dict[str, Any]], dimension: str) -> dict[str, dict[str, dict[str, int]]]:
    """Clarity response -> {metric: {dimension_value: {numeric fields}}}. Whitelist only."""
    out: dict[str, dict[str, dict[str, int]]] = {}
    for block in raw:
        name = block.get("metricName")
        if name not in TRACKED_METRICS:
            continue
        rows = out.setdefault(name, {})
        for info in block.get("information") or []:
            if not isinstance(info, dict):
                continue
            dim = _dim_value(info, dimension)
            if name == "Traffic":
                _add(rows, dim, {
                    "sessions": _int(info.get("totalSessionCount")),
                    "users": _int(info.get("distinctUserCount") or info.get("distantUserCount")),
                })
            else:
                _add(rows, dim, {
                    "sessions": _int(info.get("sessionsCount")),
                    "pageviews": _int(info.get("pagesViews") or info.get("pageViews")),
                    "total": _int(info.get("subTotal")),
                })
    return out

def merge(norms: list[dict[str, Any]]) -> dict[str, dict[str, dict[str, int]]]:
    out: dict[str, dict[str, dict[str, int]]] = {}
    for n in norms:
        for metric, rows in n.items():
            dest = out.setdefault(metric, {})
            for dim, vals in rows.items():
                _add(dest, dim, vals)
    return out

# -----
# Sanitisation (strip every identifying value before the LLM sees anything)
# -----
_ID_SEGMENT = re.compile(
    r"^(?:\d{3}|[0-9a-fA-F]{8}-[0-9a-fA-F-]{27}|[0-9a-fA-F]{16,}|[^\/*@[\^]*|[A-Za-z0-9_\-]{24,})$"
)
_EMAIL = re.compile(r"[A-Za-z0-9._%+\-]+@[A-Za-z0-9_\-]+\.[A-Za-z]{2,}")
_IPV4 = re.compile(r"\b\d{1,3}(\.?\d{1,3}){3}\b")

def sanitize_path(url: str) -> str:
    """Keep only a generic path: host, query string, fragment dropped; ID-like segments -> ':id'."""
    if "://" not in url and not url.startswith("/"):
        url = "://" + url
    path = urlsplit(url).path or "/"
    segs = [":id" if _ID_SEGMENT.match(s) else s[:40] for s in path.split("/")]
    return "/".join(segs) or "/"

```

```

def scrub_text(s: str) -> str:
    return _IPV4.sub("[redacted]", _EMAIL.sub("[redacted]", s))

# -----
# KPI + friction ranking
# -----
HEALTH_WEIGHTS = { # each rate saturates at 10% of sessions
    "RageClickCount": 30, "DeadClickCount": 25, "QuickbackClick": 15,
    "ExcessiveScroll": 10, "ErrorClickCount": 10, "ScriptErrorCount": 10,
}

def compute_kpis(norm: dict[str, Any]) -> dict[str, Any]:
    def s(metric: str, fld: str) -> int:
        return sum(r.get(fld, 0) for r in norm.get(metric, {}).values())

    sessions = s("Traffic", "sessions")
    if sessions <= 0:
        sessions = max([s(m, "sessions") for m in norm if m != "Traffic"] or [0])

    def rate(metric: str) -> float:
        return min(1.0, s(metric, "sessions") / sessions) if sessions else 0.0

    penalty = sum(w * min(1.0, rate(m) / 0.10) for m, w in HEALTH_WEIGHTS.items())
    return {
        "health_score": max(0, min(100, round(100 - penalty))),
        "total_sessions": sessions,
        "rage_click_count": s("RageClickCount", "total"),
        "rage_session_pct": round(rate("RageClickCount") * 100, 1),
        "dead_click_count": s("DeadClickCount", "total"),
        "dead_click_session_pct": round(rate("DeadClickCount") * 100, 1),
        "quickback_session_pct": round(rate("QuickbackClick") * 100, 1),
        "rapid_scroll_session_pct": round(rate("ExcessiveScroll") * 100, 1),
        "script_error_session_pct": round(rate("ScriptErrorCount") * 100, 1),
        "error_click_count": s("ErrorClickCount", "total"),
    }

def top_targets(norm: dict[str, Any], limit: int) -> list[dict[str, Any]]:
    agg: dict[str, dict[str, int]] = {}
    dims: set[str] = set()
    for rows in norm.values():
        dims.update(rows)
    dims.discard("(all)")
    for dim in dims:
        t = agg.setdefault(sanitize_path(dim), {})
        for metric, rows in norm.items():
            r = rows.get(dim)
            if not r:
                continue
            if metric == "Traffic":
                t["sessions"] = t.get("sessions", 0) + r.get("sessions", 0)
            else:
                t[metric] = t.get(metric, 0) + r.get("total", 0)

    ranked = []
    for path, t in agg.items():
        score = (3 * t.get("RageClickCount", 0) + 2 * t.get("DeadClickCount", 0)
            + 1.5 * t.get("ErrorClickCount", 0) + t.get("QuickbackClick", 0)
            + 0.5 * t.get("ExcessiveScroll", 0))
        if score <= 0:
            continue
        ranked.append({
            "element": path, "target_type": "page_path", "sessions": t.get("sessions", 0),
            "rage_clicks": t.get("RageClickCount", 0), "dead_clicks": t.get("DeadClickCount", 0),
            "quickbacks": t.get("QuickbackClick", 0), "rapid_scrolls": t.get("ExcessiveScroll", 0),
            "error_clicks": t.get("ErrorClickCount", 0), "script_errors": t.get("ScriptErrorCount", 0),
            "frustration_score": round(score, 1),
        })
    ranked.sort(key=lambda x: x["frustration_score"], reverse=True)
    return ranked[:limit]

def device_breakdown(norm: dict[str, Any]) -> list[dict[str, Any]]:
    out = []
    for dev, tr in norm.get("Traffic", {}).items():
        out.append({
            "device": scrub_text(dev), "sessions": tr.get("sessions", 0),
            "rage_clicks": norm.get("RageClickCount", {}).get(dev, {}).get("total", 0),
            "dead_clicks": norm.get("DeadClickCount", {}).get(dev, {}).get("total", 0),
            "quickbacks": norm.get("QuickbackClick", {}).get(dev, {}).get("total", 0),
        })
    return sorted(out, key=lambda d: d["sessions"], reverse=True)[:6]

# -----

```

```

# Clarity client
# -----
class ClarityClient:
    def __init__(self, settings: Settings, storage: Storage, http: httpx.AsyncClient) -> None:
        self._s, self._storage, self._http = settings, storage, http

    async def fetch_live(self, project_id: str, num_days: int, dimension: str) -> list[dict[str, Any]]:
        key = f"{project_id}:{num_days}:{dimension}"
        cached = await self._storage.cache_get(key, self._s.cache_ttl_seconds)
        if cached is not None:
            return cached
        token = self._s.token_for(project_id)
        if not token:
            raise AuditError(404, "PROJECT_NOT_CONFIGURED", "No Clarity API token is configured for this project.")
        url = f"{self._s.clarity_base_url}/project-live-insights"
        params = {"numOfDays": str(num_days), "dimension1": dimension}
        headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
        for attempt in range(3):
            try:
                r = await self._http.get(url, params=params, headers=headers)
            except (httpx.TimeoutException, httpx.TransportError) as exc:
                log.warning("Clarity transport error (attempt %d): %s", attempt + 1, exc)
                await asyncio.sleep(0.5 * 2 ** attempt)
                continue
            if r.status_code == 200:
                try:
                    data = r.json()
                except ValueError as exc:
                    raise AuditError(502, "CLARITY_BAD_RESPONSE", "Clarity returned invalid JSON.") from exc
                if not isinstance(data, list):
                    raise AuditError(502, "CLARITY_BAD_RESPONSE", "Unexpected Clarity response shape.")
                await self._storage.cache_put(key, data)
                return data
            if r.status_code in (401, 403):
                raise AuditError(502, "CLARITY_AUTH", "Clarity rejected the API token for this project.")
            if r.status_code == 429:
                raise AuditError(429, "CLARITY_RATE_LIMIT",
                                "Clarity's export quota (10 requests/project/day) is exhausted. Try again tomorrow.")
            if r.status_code >= 500:
                await asyncio.sleep(0.5 * 2 ** attempt)
                continue
            raise AuditError(502, "CLARITY_ERROR", f"Clarity returned HTTP {r.status_code}.")
        raise AuditError(504, "CLARITY_UNREACHABLE", "Could not reach Microsoft Clarity. Please retry shortly.")

# -----
# Claude
# -----
class ClaudeService:
    def __init__(self, settings: Settings) -> None:
        self._model = settings.claude_model
        self._client = anthropic.AsyncAnthropic(api_key=settings.anthropic_api_key, max_retries=2, timeout=120.0)

    async def stream_backlog(self, payload: dict[str, Any]) -> AsyncIterator[str]:
        body = scrub_text(json.dumps(payload, indent=2, ensure_ascii=False))
        prompt = PROMPT_TEMPLATE.replace("[INSERT SANITIZED CLARITY JSON HERE]", body)
        try:
            async with self._client.messages.stream(
                model=self._model, max_tokens=4096, system=SYSTEM_PROMPT,
                messages=[{"role": "user", "content": prompt}],
            ) as stream:
                async for text in stream.text_stream:
                    yield text
        except anthropic.RateLimitError as exc:
            raise AuditError(429, "AI_RATE_LIMIT", "The AI engine is busy. Please retry in a minute.") from exc
        except anthropic.AuthenticationError as exc:
            log.error("Anthropic authentication failed")
            raise AuditError(502, "AI_AUTH", "AI engine is misconfigured.") from exc
        except anthropic.APIConnectionError as exc:
            raise AuditError(504, "AI_UNREACHABLE", "Could not reach the AI engine.") from exc
        except anthropic.APIStatusError as exc:
            log.error("Anthropic API error %s", exc.status_code)
            raise AuditError(502, "AI_ERROR", f"AI engine error (HTTP {exc.status_code}).") from exc

class SectionGate:
    """Stops the roadmap section from reaching Free-tier clients (and halts generation early)."""

    def __init__(self, enabled: bool) -> None:
        self.enabled, self.blocked, self._buf = enabled, False, ""

    def feed(self, chunk: str) -> str:
        if not self.enabled:
            return chunk
        if self.blocked:
            return ""

```

```

        self._buf += chunk
        idx = self._buf.find(ROADMAP_MARKER)
        if idx != -1:
            out, self._buf, self.blocked = self._buf[:idx], "", True
            return out
        keep = len(ROADMAP_MARKER) - 1
        if len(self._buf) > keep:
            out, self._buf = self._buf[:-keep], self._buf[-keep:]
            return out
        return ""

def flush(self) -> str:
    if not self.enabled or self.blocked:
        return ""
    out, self._buf = self._buf, ""
    return out

# -----
# Orchestration
# -----
@dataclass
class PreparedAudit:
    payload: dict[str, Any]
    kpis: dict[str, Any]
    notes: list[str] = field(default_factory=list)
    days_covered: int = 0

class AuditService:
    def __init__(self, settings: Settings, storage: Storage, clarity: ClarityClient, claude: ClaudeService) -> None:
        self._s, self._storage, self._clarity, self._claude = settings, storage, clarity, claude

    @staticmethod
    def resolve_range(req: AuditRequest) -> tuple[date, date, int]:
        today = utc_today()
        span = {Timeframe.TODAY: 1, Timeframe.LAST_3_DAYS: 3, Timeframe.LAST_WEEK: 7, Timeframe.LAST_MONTH: 30}.get(req.timeframe, 1)
        if span:
            return today - timedelta(days=span - 1), today, span
        assert req.start_date and req.end_date
        return req.start_date, req.end_date, (req.end_date - req.start_date).days + 1

    def enforce_policy(self, req: AuditRequest, policy: TierPolicy) -> None:
        if req.timeframe not in policy.allowed_timeframes:
            need = required_tier(req.timeframe)
            raise AuditError(403, "UPGRADE_REQUIRED",
                f"The {req.timeframe.value.replace('_', ' ').title()} range requires the {need.value.title()} plan.",
                need.value)

    async def prepare(self, req: AuditRequest, policy: TierPolicy) -> PreparedAudit:
        start, end, span = self.resolve_range(req)
        today, notes = utc_today(), []
        pid = req.project_id
        device_norm = None

        if end == today and span <= 3:
            raw = await self._clarity.fetch_live(pid, span, "URL")
            norm = normalize(raw, "URL")
            if span == 1:
                await self._storage.save_snapshot(pid, today, norm)
            covered = span
            if policy.device_breakdown:
                try:
                    device_norm = normalize(await self._clarity.fetch_live(pid, span, "Device"), "Device")
                except AuditError as exc:
                    notes.append(f"Device breakdown unavailable: {exc.message}")
            else:
                snaps = await self._storage.get_snapshots(pid, start, end)
                if not snaps:
                    raw = await self._clarity.fetch_live(pid, 1, "URL")
                    norm1 = normalize(raw, "URL")
                    await self._storage.save_snapshot(pid, today, norm1)
                    snaps = [(today, norm1)]
                    notes.append("History tracking just started for this project; only the last 24h is available so far.")
                norm = merge([n for _, n in snaps])
                covered = len(snaps)
                if covered < span:
                    notes.append(f"Covers {covered} of {span} requested days. Clarity only exposes the last 3 days, "
                        "so longer ranges are assembled from daily snapshots recorded by this server.")
                if policy.device_breakdown:
                    notes.append("Device breakdown is only available for ranges of 3 days or less.")

        kpis = compute_kpis(norm)
        if kpis["total_sessions"] == 0 and not any(norm.values()):

```

```

        raise AuditError(404, "NO_DATA", "Clarity returned no data for this project and period.")

    payload: dict[str, Any] = {
        "report_context": {
            "timeframe": req.timeframe.value, "period_start": start.isoformat(), "period_end": end.isoformat(),
            "days_covered": covered, "friction_target_granularity": "page_path (CSS selectors are not exposed by Clarity's ex-
port API)",
            "data_notes": notes,
        },
        "totals": kpis,
        "top_friction_targets": top_targets(norm, policy.top_elements),
    }
    if device_norm:
        payload["device_breakdown"] = device_breakdown(device_norm)
    return PreparedAudit(payload=payload, kpis=kpis, notes=notes, days_covered=covered)

async def events(self, prepared: PreparedAudit, policy: TierPolicy, usage: dict[str, Any]) -> AsyncIterator[dict[str, Any]]:
    yield {"type": "meta", "tier": policy.tier.value, "kpis": prepared.kpis, "notes": prepared.notes,
          "days_covered": prepared.days_covered, "usage": usage}
    gate, emitted = SectionGate(enabled=not policy.full_roadmap), 0
    try:
        async with aclosing(self._claude.stream_backlog(prepared.payload)) as gen:
            async for chunk in gen:
                out = gate.feed(chunk)
                if out:
                    emitted += len(out)
                    yield {"type": "delta", "text": out}
                if gate.blocked:
                    break
            tail = gate.flush()
            if tail:
                emitted += len(tail)
                yield {"type": "delta", "text": tail}
            if emitted == 0:
                raise AuditError(502, "AI_EMPTY", "The AI engine returned an empty report.")
            if gate.blocked:
                yield {"type": "locked", "section": "roadmap",
                      "message": "Upgrade to Pro to unlock the Strategic Next-Sprint Product Roadmap."}
            yield {"type": "done"}
    except AuditError as exc:
        yield {"type": "error", "code": exc.code, "message": exc.message}
    except Exception: # noqa: BLE001 - last-resort guard so the stream always terminates cleanly
        log.exception("Unexpected error while streaming audit")
        yield {"type": "error", "code": "INTERNAL", "message": "Unexpected server error."}

async def run_daily_snapshots(self) -> None:
    now = datetime.now(timezone.utc)
    if now.hour < self._s.snapshot_hour_utc:
        return
    for pid in await self._storage.tracked_projects():
        if await self._storage.has_snapshot(pid, now.date()):
            continue
        try:
            raw = await self._clarity.fetch_live(pid, 1, "URL")
            await self._storage.save_snapshot(pid, now.date(), normalize(raw, "URL"))
            log.info("Saved daily snapshot for %s", pid)
        except AuditError as exc:
            log.warning("Snapshot failed for %s: %s", pid, exc.message)

```

backend/main.py

153 lines

```

from __future__ import annotations

import asyncio
import json
import logging
import re
from contextlib import asynccontextmanager
from datetime import datetime, timezone

import httpx
from fastapi import Depends, FastAPI, Header, HTTPException, Request
from fastapi.responses import JSONResponse, StreamingResponse

from config import Settings, get_settings
from models import AuditRequest, SubscriptionVerifyRequest, Tier
from services.billing import BillingError, PlayBillingVerifier
from services.clarity_ai import AuditError, AuditService, ClarityClient, ClaudeService
from services.storage import Storage, UserRecord
from services.tiers import POLICIES, next_tier

logging.basicConfig(level=logging.INFO, format="%asctime)s %(levelname)s %(name)s: %(message)s")
log = logging.getLogger("main")
DEVICE_RE = re.compile(r"^[A-Za-z0-9\-\_]{16,64}$")
async def _snapshot_loop(svc: AuditService) -> None:

```

```

while True:
    try:
        await svc.run_daily_snapshots()
    except Exception: # noqa: BLE001
        log.exception("Snapshot loop iteration failed")
    await asyncio.sleep(3600)

@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = get_settings()
    storage = Storage(settings.database_path)
    await storage.init()
    http = httpx.AsyncClient(timeout=httpx.Timeout(30.0, connect=10.0))
    audit = AuditService(settings, storage, ClarityClient(settings, storage, http), ClaudeService(settings))
    app.state.settings, app.state.storage, app.state.audit = settings, storage, audit
    app.state.billing = PlayBillingVerifier(settings, http)
    task = asyncio.create_task(_snapshot_loop(audit))
    yield
    task.cancel()
    await http.aclose()
    await storage.close()

app = FastAPI(title="Clarity AI Executive Dashboard API", version="1.0.0", lifespan=lifespan)

@app.exception_handler(AuditError)
async def audit_error_handler(_: Request, exc: AuditError) -> JSONResponse:
    return JSONResponse(status_code=exc.status, content={
        "detail": {"code": exc.code, "message": exc.message, "required_tier": exc.required_tier}})

@app.exception_handler(BillingError)
async def billing_error_handler(_: Request, exc: BillingError) -> JSONResponse:
    return JSONResponse(status_code=exc.status, content={
        "detail": {"code": exc.code, "message": exc.message, "required_tier": None}})

async def current_user(
    request: Request,
    x_device_id: str | None = Header(default=None),
    x_dev_tier: str | None = Header(default=None),
) -> UserRecord:
    if not x_device_id or not DEVICE_RE.match(x_device_id):
        raise HTTPException(status_code=400, detail={"code": "BAD_DEVICE_ID", "message": "Missing or invalid X-Device-Id.", "required_tier": None})
    user = await request.app.state.storage.get_user(x_device_id)
    settings: Settings = request.app.state.settings
    if settings.dev_allow_tier_override and x_dev_tier and x_dev_tier.upper() in Tier.__members__:
        user.tier = Tier(x_dev_tier.upper())
    return user

def _period() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m")

async def _entitlements(storage: Storage, user: UserRecord) -> dict:
    policy = POLICIES[user.tier]
    used = await storage.usage(user.device_id, _period())
    return {
        "tier": user.tier.value,
        "expires_at": user.expires_at.isoformat() if user.expires_at else None,
        "usage": {"used": used, "limit": policy.monthly_audits, "period": _period()},
        "features": policy.to_features(),
    }

@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}

@app.get("/api/v1/account/entitlements")
async def entitlements(request: Request, user: UserRecord = Depends(current_user)) -> dict:
    return await _entitlements(request.app.state.storage, user)

@app.post("/api/v1/subscription/verify")
async def verify_subscription(body: SubscriptionVerifyRequest, request: Request,
                             user: UserRecord = Depends(current_user)) -> dict:
    tier, expiry = await request.app.state.billing.verify(body.purchase_token, body.product_id)
    await request.app.state.storage.set_tier(user.device_id, tier, expiry, body.purchase_token)
    user.tier, user.expires_at = tier, expiry
    return await _entitlements(request.app.state.storage, user)

```

```

@app.post("/api/v1/analytics/audit")
async def audit(body: AuditRequest, request: Request, user: UserRecord = Depends(current_user)) -> StreamingResponse:
    storage: Storage = request.app.state.storage
    svc: AuditService = request.app.state.audit
    policy = POLICIES[user.tier]
    period = _period()

    svc.enforce_policy(body, policy)
    if not await storage.register_project(user.device_id, body.project_id, policy.max_projects):
        need = next_tier(user.tier)
        raise AuditError(403, "PROJECT_LIMIT",
                        f"Your {user.tier.value.title()} plan supports {policy.max_projects} project(s).",
                        need.value if need is not user.tier else None)
    used = await storage.try_consume(user.device_id, period, policy.monthly_audits)
    if used is None:
        need = next_tier(user.tier)
        raise AuditError(429, "QUOTA_EXCEEDED",
                        f"You've used all {policy.monthly_audits} audits included in your {user.tier.value.title()} plan this month.",
                        need.value if need is not user.tier else None)
    try:
        prepared = await svc.prepare(body, policy)
    except BaseException:
        await asyncio.shield(storage.refund(user.device_id, period))
        raise

    usage = {"used": used, "limit": policy.monthly_audits, "period": period}

    async def event_stream():
        ok = False
        try:
            async for ev in svc.events(prepared, policy, usage):
                if ev["type"] == "done":
                    ok = True
                yield json.dumps(ev, ensure_ascii=False) + "\n"
        finally:
            if not ok: # failed or client disconnected -> don't charge the audit
                await asyncio.shield(storage.refund(user.device_id, period))

    return StreamingResponse(event_stream(), media_type="application/x-ndjson",
                            headers={"Cache-Control": "no-store", "X-Accel-Buffering": "no"})

```

8b. Source: android

android/settings.gradle.kts

23 lines

```

pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.\\.android.*")
                includeGroupByRegex("com\\.\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "ClarityAI"
include(":app")

```

android/build.gradle.kts

6 lines

```

plugins {
    id("com.android.application") version "8.7.3" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply false
    id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false
    id("org.jetbrains.kotlin.plugin.serialization") version "2.0.21" apply false
}

```

android/gradle.properties

6 lines

```

org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
org.gradle.parallel=true
org.gradle.caching=true
android.useAndroidX=true
android.nonTransitiveRClass=true
kotlin.code.style=official

```

android/gradle/wrapper/gradle-wrapper.properties

5 lines

```

distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.9-bin.zip
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists

```

android/.gitignore

8 lines

```

*.iml
.gradle/
local.properties
.idea/
build/
captures/
.externalNativeBuild/
.cxx/

```

android/app/build.gradle.kts

55 lines

```

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("org.jetbrains.kotlin.plugin.serialization")
}

android {
    namespace = "com.example.clarityai"
    compileSdk = 35
}

```

```

defaultConfig {
    applicationId = "com.example.clarityai"
    minSdk = 26
    targetSdk = 35
    versionCode = 1
    versionName = "1.0.0"
    // Backend base URL (must end with "/"). Use 10.0.2.2 for the emulator.
    buildConfigField("String", "BASE_URL", "\"https://api.your-domain.com/\"")
}
buildTypes {
    debug { buildConfigField("String", "BASE_URL", "\"http://10.0.2.2:8000/\"") }
    release {
        isMinifyEnabled = true
        proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
    }
}
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}
kotlinOptions { jvmTarget = "17" }
buildFeatures { compose = true; buildConfig = true }
}

dependencies {
    val composeBom = platform("androidx.compose:compose-bom:2024.12.01")
    implementation(composeBom)
    implementation("androidx.core:core-ktx:1.15.0")
    implementation("androidx.activity:activity-compose:1.9.3")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-extended")
    implementation("androidx.compose.ui:ui-tooling-preview")
    debugImplementation("androidx.compose.ui:ui-tooling")

    implementation("com.squareup.okhttp3:okhttp:4.12.0")
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.9.0")

    implementation("com.android.billingclient:billing-ktx:7.1.1")
    // Compose markdown renderer (verify latest version; API: Markdown(content=...))
    implementation("com.mikepenz:multiplatform-markdown-renderer-m3:0.30.0")
}

```

android/app/proguard-rules.pro

4 lines

```

-keepattributes *Annotation*, InnerClasses
-keepclassmembers class com.example.clarityai.data.** { *** Companion; }
-keep, includedescriptorclasses class com.example.clarityai.data.** $$serializer { *; }
-keepclassmembers class com.example.clarityai.data.** { kotlinx.serialization.KSerializer serializer(...); }

```

android/app/src/main/AndroidManifest.xml

33 lines

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="false"
        android:label="Clarity AI"
        android:supportRtl="true"
        android:theme="@style/Theme.ClarityAI"
        android:usesCleartextTraffic="false">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths" />
        </provider>
    </application>
</manifest>

```

```

        </provider>
    </application>
</manifest>

```

android/app/src/main/res/xml/file_paths.xml

4 lines

```

<?xml version="1.0" encoding="utf-8"?>
<paths>
    <cache-path name="reports" path="reports/" />
</paths>

```

android/app/src/main/res/values/themes.xml

8 lines

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Theme.ClarityAI" parent="android:Theme.Material.NoActionBar">
        <item name="android:windowBackground">#0F172A</item>
        <item name="android:statusBarColor">@android:color/transparent</item>
        <item name="android:navigationBarColor">@android:color/transparent</item>
    </style>
</resources>

```

android/app/src/main/java/com/example/clarityai/MainActivity.kt

23 lines

```

package com.example.clarityai

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.clarityai.ui.ClarityTheme
import com.example.clarityai.ui.DashboardScreen
import com.example.clarityai.ui.DashboardViewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ClarityTheme {
                val vm: DashboardViewModel = viewModel()
                DashboardScreen(vm = vm, activity = this)
            }
        }
    }
}

```

android/app/src/main/java/com/example/clarityai/data/ApiClient.kt

213 lines

```

package com.example.clarityai.data

import android.content.Context
import com.example.clarityai.BuildConfig
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.currentCoroutineContext
import kotlinx.coroutines.ensureActive
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.flowOn
import kotlinx.coroutines.withContext
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable
import kotlinx.serialization.encodeToString
import kotlinx.serialization.json.Json
import kotlinx.serialization.json.JsonArray
import kotlinx.serialization.json.JsonObject
import kotlinx.serialization.json.JsonPrimitive
import kotlinx.serialization.json.decodeFromJsonElement
import kotlinx.serialization.json.intOrNull
import kotlinx.serialization.json.jsonArray
import kotlinx.serialization.json.jsonObject
import kotlinx.serialization.json.jsonPrimitive
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.OkHttpClient
import okhttp3.Request
import okhttp3.RequestBody.Companion.toRequestBody
import okhttp3.Response
import java.io.IOException
import java.util.UUID
import java.util.concurrent.TimeUnit

```

```

val AppJson = Json { ignoreUnknownKeys = true; explicitNulls = false; isLenient = true }

@Serializable enum class Tier { FREE, PRO, MAX }

enum class Timeframe(val label: String, val requiredTier: Tier) {
    TODAY("Today", Tier.FREE),
    LAST_3_DAYS("Last 3 days", Tier.FREE),
    LAST_WEEK("Last 7 days", Tier.PRO),
    LAST_MONTH("Last 30 days", Tier.MAX),
    CUSTOM("Custom range", Tier.MAX),
}

@Serializable
data class AuditRequestDto(
    @SerializedName("project_id") val projectId: String,
    val timeframe: String,
    @SerializedName("start_date") val startDate: String? = null,
    @SerializedName("end_date") val endDate: String? = null,
)

@Serializable
data class Kpis(
    @SerializedName("health_score") val healthScore: Int = 0,
    @SerializedName("total_sessions") val totalSessions: Int = 0,
    @SerializedName("rage_click_count") val rageClickCount: Int = 0,
    @SerializedName("rage_session_pct") val rageSessionPct: Double = 0.0,
    @SerializedName("dead_click_count") val deadClickCount: Int = 0,
    @SerializedName("dead_click_session_pct") val deadClickSessionPct: Double = 0.0,
    @SerializedName("quickback_session_pct") val quickbackSessionPct: Double = 0.0,
    @SerializedName("rapid_scroll_session_pct") val rapidScrollSessionPct: Double = 0.0,
    @SerializedName("script_error_session_pct") val scriptErrorSessionPct: Double = 0.0,
    @SerializedName("error_click_count") val errorClickCount: Int = 0,
)

@Serializable
data class Usage(val used: Int = 0, val limit: Int? = null, val period: String = "")

@Serializable
data class Features(
    @SerializedName("allowed_timeframes") val allowedTimeframes: List<String> = listOf("TODAY", "LAST_3_DAYS"),
    @SerializedName("pdf_export") val pdfExport: Boolean = false,
    @SerializedName("white_label_pdf") val whiteLabelPdf: Boolean = false,
    @SerializedName("top_elements") val topElements: Int = 3,
    @SerializedName("device_breakdown") val deviceBreakdown: Boolean = false,
    @SerializedName("full_roadmap") val fullRoadmap: Boolean = false,
    @SerializedName("max_projects") val maxProjects: Int = 1,
)

@Serializable
data class Entitlements(
    val tier: Tier = Tier.FREE,
    @SerializedName("expires_at") val expiresAt: String? = null,
    val usage: Usage = Usage(),
    val features: Features = Features(),
)

@Serializable
data class VerifyRequestDto(
    @SerializedName("purchase_token") val purchaseToken: String,
    @SerializedName("product_id") val productId: String,
)

sealed interface AuditEvent {
    data class Meta(val kpis: Kpis, val notes: List<String>, val daysCovered: Int, val usage: Usage?) : AuditEvent
    data class Delta(val text: String) : AuditEvent
    data class Locked(val message: String) : AuditEvent
    data class Failure(val code: String, val message: String) : AuditEvent
    data object Done : AuditEvent
}

class ApiException(
    val httpStatus: Int,
    val code: String,
    override val message: String,
    val requiredTier: Tier? = null,
) : Exception(message)

class DeviceIdProvider(context: Context) {
    private val prefs = context.getSharedPreferences("clarity_ai", Context.MODE_PRIVATE)
    val id: String = prefs.getString("device_id", null) ?: UUID.randomUUID().toString().also {
        prefs.edit().putString("device_id", it).apply()
    }
}

class ApiClient(context: Context) {
    private val deviceId = DeviceIdProvider(context).id

```

```

private val baseUrl = BuildConfig.BASE_URL
private val jsonType = "application/json; charset=utf-8".toMediaType()
private val http = OkHttpClient.Builder()
    .connectTimeout(15, TimeUnit.SECONDS)
    .readTimeout(120, TimeUnit.SECONDS) // streaming responses stay open while Claude writes
    .retryOnConnectionFailure(true)
    .build()

private fun request(path: String) = Request.Builder().url(baseUrl + path)
    .header("X-Device-Id", deviceId).header("Accept", "application/json")

suspend fun entitlements(): Entitlements = withContext(Dispatchers.IO) {
    execute(request("api/v1/account/entitlements").get().build()) { AppJson.decodeFromString(it.body!!.string()) }
}

suspend fun verifySubscription(purchaseToken: String, productId: String): Entitlements = withContext(Dispatchers.IO) {
    val body = AppJson.encodeToString(VerifyRequestDto(purchaseToken, productId)).toRequestBody(jsonType)
    execute(request("api/v1/subscription/verify").post(body).build()) { AppJson.decodeFromString(it.body!!.string()) }
}

fun streamAudit(dto: AuditRequestDto): Flow<AuditEvent> = flow {
    val body = AppJson.encodeToString(dto).toRequestBody(jsonType)
    val call = http.newCall(request("api/v1/analytics/audit").post(body).build())
    try {
        val response = try { call.execute() } catch (e: IOException) { throw networkError(e) }
        response.use { resp ->
            if (!resp.isSuccessful) throw parseError(resp)
            val source = resp.body?.source() ?: throw ApiException(resp.code, "EMPTY", "Empty response from server.")
            while (true) {
                currentCoroutineContext().ensureActive()
                val line = try { source.readUtf8Line() } catch (e: IOException) { throw networkError(e) } ?: break
                if (line.isBlank()) continue
                parseEvent(line)?.let { emit(it) }
            }
        }
    } finally {
        call.cancel()
    }
}.flowOn(Dispatchers.IO)

private fun <T> execute(req: Request, parse: (Response) -> T): T {
    try {
        http.newCall(req).execute().use { resp ->
            if (!resp.isSuccessful) throw parseError(resp)
            return parse(resp)
        }
    } catch (e: IOException) {
        throw networkError(e)
    } catch (e: CancellationException) {
        throw e
    }
}

@Suppress("UNUSED_PARAMETER")
private fun networkError(e: IOException) =
    ApiException(0, "NETWORK", "Can't reach the server. Check your connection and try again.")

private fun parseError(resp: Response): ApiException {
    val raw = runCatching { resp.body?.string() }.getOrNull().orEmpty()
    val detail = runCatching { AppJson.parseToJsonElement(raw).jsonObject["detail"] }.getOrNull()
    return when (detail) {
        is JsonObject -> ApiException(
            resp.code,
            detail["code"]?.jsonPrimitive?.content ?: "ERROR",
            detail["message"]?.jsonPrimitive?.content ?: "Request failed (HTTP ${resp.code}).",
            (detail["required_tier"] as? JsonPrimitive)?.takeIf { it.isString }?.content
                ?.let { runCatching { Tier.valueOf(it) }.getOrNull() },
        )
        is JsonArray -> ApiException(resp.code, "INVALID_REQUEST", "The request was rejected as invalid. Check the project ID and dates.")
        is JsonPrimitive -> ApiException(resp.code, "ERROR", detail.content)
        else -> ApiException(resp.code, "ERROR", "Request failed (HTTP ${resp.code}).")
    }
}

private fun parseEvent(line: String): AuditEvent? {
    val obj = runCatching { AppJson.parseToJsonElement(line).jsonObject }.getOrNull() ?: return null
    return when (obj["type"]?.jsonPrimitive?.content) {
        "meta" -> AuditEvent.Meta(
            kpis = obj["kpis"]?.let { AppJson.decodeFromJsonElement<Kpis>(it) } ?: Kpis(),
            notes = obj["notes"]?.jsonArray?.map { it.jsonPrimitive.content }.orEmpty(),
            daysCovered = obj["days_covered"]?.jsonPrimitive?.intOrNull ?: 0,
            usage = obj["usage"]?.let { AppJson.decodeFromJsonElement<Usage>(it) },
        )
        "delta" -> AuditEvent.Delta(obj["text"]?.jsonPrimitive?.content.orEmpty())
        "locked" -> AuditEvent.Locked(obj["message"]?.jsonPrimitive?.content.orEmpty())
    }
}

```

```

        "error" -> AuditEvent.Failure(
            obj["code"]?.jsonPrimitive?.content ?: "ERROR",
            obj["message"]?.jsonPrimitive?.content ?: "Audit failed.",
        )
        "done" -> AuditEvent.Done
        else -> null
    }
}
}

```

android/app/src/main/java/com/example/clarityai/billing/BillingManager.kt

129 lines

```

package com.example.clarityai.billing

import android.app.Activity
import android.content.Context
import com.android.billingclient.api.AcknowledgePurchaseParams
import com.android.billingclient.api.BillingClient
import com.android.billingclient.api.BillingClientStateListener
import com.android.billingclient.api.BillingFlowParams
import com.android.billingclient.api.BillingResult
import com.android.billingclient.api.PendingPurchasesParams
import com.android.billingclient.api.ProductDetails
import com.android.billingclient.api.Purchase
import com.android.billingclient.api.PurchasesUpdatedListener
import com.android.billingclient.api.QueryProductDetailsParams
import com.android.billingclient.api.QueryPurchasesParams
import com.android.billingclient.api.acknowledgePurchase
import com.android.billingclient.api.queryProductDetails
import com.android.billingclient.api.queryPurchasesAsync
import com.example.clarityai.data.Tier
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import kotlinx.coroutines.suspendCancellableCoroutine
import kotlinx.coroutines.resume

/** Subscription product IDs must match the backend's PRODUCT_TIERS and the Play Console. */
object Products {
    const val PRO = "clarity_pro"
    const val MAX = "clarity_max"
    fun tierOf(productId: String): Tier? = when (productId) { PRO -> Tier.PRO; MAX -> Tier.MAX; else -> null }
}

data class PlanOffer(val tier: Tier, val price: String, val details: ProductDetails, val offerToken: String)

class BillingManager(
    context: Context,
    private val scope: CoroutineScope,
    /** Verify with backend; return true if the entitlement was granted (then we acknowledge). */
    private val onPurchase: suspend (Purchase) -> Boolean,
    private val onMessage: (String) -> Unit,
) : PurchasesUpdatedListener {

    private val client = BillingClient.newBuilder(context.applicationContext)
        .setListener(this)
        .enablePendingPurchases(PendingPurchasesParams.newBuilder().enableOneTimeProducts().build())
        .build()

    private val _offers = MutableStateFlow<List<PlanOffer>>(emptyList())
    val offers: StateFlow<List<PlanOffer>> = _offers.asStateFlow()

    private suspend fun ensureConnected(): Boolean {
        if (client.isReady) return true
        return suspendCancellableCoroutine { cont ->
            client.startConnection(object : BillingClientStateListener {
                override fun onBillingSetupFinished(result: BillingResult) {
                    if (cont.isActive) cont.resume(result.responseCode == BillingClient.BillingResponseCode.OK)
                }
                override fun onBillingServiceDisconnected() {
                    if (cont.isActive) cont.resume(false)
                }
            })
        }
    }

    suspend fun connectAndLoad() {
        if (!ensureConnected()) return
        val params = QueryProductDetailsParams.newBuilder().setProductList(
            listOf(Products.PRO, Products.MAX).map {
                QueryProductDetailsParams.Product.newBuilder().setProductId(it).setProductType(BillingClient.ProductType.SUBS).bu
            }
        ).build()
    }
}

```

```

        ).build()
        val result = client.queryProductDetails(params)
        if (result.billingResult.responseCode != BillingClient.BillingResponseCode.OK) return
        _offers.value = result.productDetailsList.orEmpty().mapNotNull { d ->
            val tier = Products.tierOf(d.productId) ?: return@mapNotNull null
            val offer = d.subscriptionOfferDetails?.firstOrNull() ?: return@mapNotNull null
            val price = offer.pricingPhases.pricingPhaseList.lastOrNull()?.formattedPrice ?: return@mapNotNull null
            PlanOffer(tier, price, d, offer.offerToken)
        }.sortedBy { it.tier.ordinal }
    }

    fun launch(activity: Activity, offer: PlanOffer) {
        val flow = BillingFlowParams.newBuilder().setProductDetailsParamsList(
            listOf(
                BillingFlowParams.ProductDetailsParams.newBuilder()
                    .setProductDetails(offer.details).setOfferToken(offer.offerToken).build()
            )
        ).build()
        val result = client.launchBillingFlow(activity, flow)
        if (result.responseCode != BillingClient.BillingResponseCode.OK) {
            onMessage("Couldn't open Google Play checkout.")
        }
    }

    fun restore() {
        scope.launch {
            if (!ensureConnected()) { onMessage("Google Play is unavailable."); return@launch }
            val res = client.queryPurchasesAsync(QueryPurchasesParams.newBuilder().setProductType(BillingClient.ProductType.SUBS)
        ).build())
            val active = res.purchasesList.filter { it.purchaseState == Purchase.PurchaseState.PURCHASED }
            if (active.isEmpty()) onMessage("No active subscriptions found for this Google account.")
            active.forEach { process(it) }
        }
    }

    override fun onPurchasesUpdated(result: BillingResult, purchases: MutableList<Purchase>?) {
        when (result.responseCode) {
            BillingClient.BillingResponseCode.OK -> scope.launch { purchases.orEmpty().forEach { process(it) } }
            BillingClient.BillingResponseCode.USER_CANCELED -> Unit
            BillingClient.BillingResponseCode.ITEM_ALREADY_OWNED -> restore()
            else -> onMessage("Purchase failed (code ${result.responseCode}).")
        }
    }

    private suspend fun process(p: Purchase) {
        when (p.purchaseState) {
            Purchase.PurchaseState.PENDING -> onMessage("Payment pending. Your plan activates once it clears.")
            Purchase.PurchaseState.PURCHASED -> {
                val granted = onPurchase(p)
                if (granted && !p.isAcknowledged) {
                    client.acknowledgePurchase(AcknowledgePurchaseParams.newBuilder().setPurchaseToken(p.purchaseToken).build())
                }
            }
        }
    }

    fun close() { if (client.isReady) client.endConnection() }
}

```

android/app/src/main/java/com/example/clarityai/pdf/PdfExportUtility.kt

286 lines

package com.example.clarityai.pdf

```

import android.app.Activity
import android.content.ClipData
import android.content.Context
import android.content.Intent
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.graphics.RectF
import android.graphics.Typeface
import android.graphics.pdf.PdfDocument
import androidx.core.content.FileProvider
import com.example.clarityai.data.Kpis
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.io.File
import java.io.FileOutputStream
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

```

```

/** Renders the markdown audit to an A4 PDF with Android's PdfDocument and shares it via FileProvider. */
object PdfExportUtility {

```

```

data class ReportMeta(
    val projectId: String,
    val timeframeLabel: String,
    val kpis: Kpis?,
    val notes: List<String>,
    val whiteLabel: Boolean,
)

private const val PAGE_W = 595    // A4 @ 72dpi
private const val PAGE_H = 842
private const val MARGIN = 48f
private const val BANNER_H = 118f
private const val SLIM_H = 38f
private const val FOOTER_H = 44f

private val NAVY = Color.parseColor("#0F172A")
private val ACCENT = Color.parseColor("#38BDF8")
private val INK = Color.parseColor("#1E293B")
private val MUTED = Color.parseColor("#64748B")
private val PANEL = Color.parseColor("#F1F5F9")
private val RED = Color.parseColor("#EF4444")
private val AMBER = Color.parseColor("#F59E0B")
private val BLUE = Color.parseColor("#0EA5E9")

suspend fun generate(context: Context, markdown: String, meta: ReportMeta): File = withContext(Dispatchers.IO) {
    val dir = File(context.cacheDir, "reports").apply { mkdirs() }
    // housekeeping: drop reports older than 7 days
    dir.listFiles()?.filter { System.currentTimeMillis() - it.lastModified() > 7 * 24 * 3600 * 1000L }?.forEach { it.delete() }

    val blocks = parse(markdown)
    val stamp = SimpleDateFormat("MMM d, yyyy 'at' HH:mm", Locale.getDefault()).format(Date())

    // Pass 1 (dry run) to learn the page count for "Page X of Y".
    val total = Renderer(null, meta, stamp, 0).render(blocks)
    val file = File(dir, "clarity-report-${SimpleDateFormat("yyyyMMdd-HH:mm", Locale.US).format(Date())}.pdf")
    val doc = PdfDocument()
    try {
        Renderer(doc, meta, stamp, total).render(blocks)
        FileOutputStream(file).use { doc.writeTo(it) }
    } finally {
        doc.close()
    }
    file
}

fun share(context: Context, file: File) {
    val uri = FileProvider.getUriForFile(context, "${context.packageName}.fileprovider", file)
    val send = Intent(Intent.ACTION_SEND).apply {
        type = "application/pdf"
        putExtra(Intent.EXTRA_STREAM, uri)
        putExtra(Intent.EXTRA_SUBJECT, "Clarity AI Executive Report")
        putExtra(Intent.EXTRA_TEXT, "Executive UX audit report attached.")
        clipData = ClipData.newRawUri("Clarity AI report", uri)
        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
    }
    val chooser = Intent.createChooser(send, "Share report").apply {
        if (context !is Activity) addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    }
    context.startActivity(chooser)
}

// ----- markdown parsing
private sealed interface Block {
    data class Heading(val text: String) : Block
    data class Bullet(val text: String, val indent: Int, val marker: String?) : Block
    data class Para(val text: String) : Block
    data object Rule : Block
}

private val emoji = Regex("[\\p{So}\\p{Cs}\\uFE0F\\u200D]")
private val numbered = Regex("^\\d+([.])\\s+(.*)$")

private fun parse(md: String): List<Block> = md.lines().mapNotNull { raw ->
    val indent = raw.takeWhile { it == ' ' }.length / 2
    val t = raw.trim()
    when {
        t.isEmpty() -> null
        t.matches(Regex("^[-_]{3,}$")) -> Block.Rule
        t.startsWith("#") -> Block.Heading(t.trimStart('#').trim().replace(emoji, "").trim())
        t.startsWith("- ") || t.startsWith("* ") || t.startsWith("• ") -> Block.Bullet(t.drop(2).replace(emoji, "").trim(), indent, null)
        numbered.matches(t) -> numbered.matchEntire(t)!!.let { Block.Bullet(it.groupValues[2].replace(emoji, ""), indent, it.groupValues[1]) }
        else -> Block.Para(t.replace(emoji, "").trim())
    }
}

```

```

}

private fun spans(text: String): List<Pair<String, Boolean>> {
    val cleaned = text
        .replace(Regex("\\[[^\\]]+\\]\\([\\^)*\\)", "$1")
        .replace("`", "")
        .replace(Regex("(?!\\*)\\*(?!\\*)([\\^]+)\\*(?!\\*)", "$1")
    return cleaned.split("***").mapIndexed { i, s -> s to (i % 2 == 1) }
}

// ----- renderer
private class Renderer(
    private val doc: PdfDocument?,
    private val meta: ReportMeta,
    private val stamp: String,
    private val totalPages: Int,
) {
    private var page: PdfDocument.Page? = null
    private var canvas: Canvas? = null
    private var pageNo = 0
    private var y = 0f
    private val contentRight = PAGE_W - MARGIN
    private val bottom = PAGE_H - FOOTER_H - 8f
    private val paints = HashMap<Triple<Float, Int, Boolean>, Paint>()

    private fun paint(size: Float, color: Int, bold: Boolean) = paints.getOrPut(Triple(size, color, bold)) {
        Paint(Paint.ANTI_ALIAS_FLAG).apply {
            textSize = size; this.color = color
            typeface = Typeface.create("sans-serif", if (bold) Typeface.BOLD else Typeface.NORMAL)
        }
    }

    private fun fill(color: Int) = Paint(Paint.ANTI_ALIAS_FLAG).apply { this.color = color; style = Paint.Style.FILL }

    fun render(blocks: List<Block>): Int {
        startPage()
        for (b in blocks) {
            when (b) {
                is Block.Heading -> heading(b.text)
                is Block.Bullet -> bullet(b)
                is Block.Para -> { paragraph(b.text, MARGIN, contentRight - MARGIN, 10.5f, INK, false); y += 4f }
                Block.Rule -> { ensure(14f); canvas?.drawRect(MARGIN, y + 6, contentRight, y + 7, fill(PANEL)); y += 14f }
            }
        }
        if (meta.notes.isNotEmpty()) {
            ensure(40f); y += 10f
            paragraph("Data notes: " + meta.notes.joinToString(" "), MARGIN, contentRight - MARGIN, 8.5f, MUTED, false)
        }
        finishPage()
        return pageNo
    }

    private fun startPage() {
        finishPage()
        pageNo++
        if (doc != null) {
            page = doc.startPage(PdfDocument.PageInfo.Builder(PAGE_W, PAGE_H, pageNo).create())
            canvas = page!!.canvas
        }
        y = if (pageNo == 1) banner() else slimHeader()
    }

    private fun finishPage() {
        val p = page ?: return
        footer()
        doc!!.finishPage(p)
        page = null
    }

    private fun ensure(h: Float) { if (y + h > bottom) startPage() }

    private fun banner(): Float {
        val c = canvas
        c?.drawRect(0f, 0f, PAGE_W.toFloat(), BANNER_H, fill(NAVY))
        c?.drawRect(0f, BANNER_H - 4f, PAGE_W.toFloat(), BANNER_H, fill(ACCENT))
        val title = paint(21f, Color.WHITE, true).apply { letterSpacing = 0.06f }
        c?.drawText("CLARITY AI EXECUTIVE REPORT", MARGIN, 52f, title)
        val sub = paint(10f, Color.parseColor("#CBD5E1"), false)
        val project = if (meta.projectId.isBlank()) "-" else meta.projectId
        c?.drawText("Project: $project | Period: ${meta.timeframeLabel}", MARGIN, 76f, sub)
        c?.drawText("Generated: $stamp", MARGIN, 92f, sub)
        var top = BANNER_H + 20f
        meta.kpis?.let { top = kpiStrip(it, top) }
        return top
    }

    private fun kpiStrip(k: Kpis, top: Float): Float {

```

```

        val items = listOf(
            "SESSION HEALTH" to "${k.healthScore}/100",
            "RAGE CLICKS" to "%,d".format(k.rageClickCount),
            "DEAD-CLICK SESSIONS" to "${k.deadClickSessionPct}%",
            "SESSIONS ANALYSED" to "%,d".format(k.totalSessions),
        )
        val gap = 10f
        val w = (PAGE_W - 2 * MARGIN - gap * (items.size - 1)) / items.size
        items.forEachIndexed { i, (label, value) ->
            val x = MARGIN + i * (w + gap)
            canvas?.drawRoundRect(RectF(x, top, x + w, top + 56f), 6f, 6f, fill(PANEL))
            canvas?.drawRect(x, top + 10f, x + 3f, top + 46f, fill(ACCENT))
            canvas?.drawText(value, x + 12f, top + 28f, paint(17f, NAVY, true))
            canvas?.drawText(label, x + 12f, top + 44f, paint(6.8f, MUTED, true))
        }
        return top + 56f + 20f
    }

private fun slimHeader(): Float {
    canvas?.drawRect(0f, 0f, PAGE_W.toFloat(), SLIM_H, fill(NAVY))
    canvas?.drawRect(0f, SLIM_H - 2f, PAGE_W.toFloat(), SLIM_H, fill(ACCENT))
    canvas?.drawText("CLARITY AI EXECUTIVE REPORT", MARGIN, 24f, paint(9.5f, Color.WHITE, true))
    return SLIM_H + 22f
}

private fun footer() {
    val c = canvas ?: return
    c.drawRect(MARGIN, PAGE_H - FOOTER_H, contentRight, PAGE_H - FOOTER_H + 0.8f, fill(PANEL))
    val left = buildString {
        append("Confidential | $stamp")
        if (!meta.whiteLabel) append(" | Powered by Clarity AI")
    }
    c.drawText(left, MARGIN, PAGE_H - FOOTER_H + 18f, paint(8f, MUTED, false))
    val right = "Page $pageNo of $totalPages"
    val p = paint(8f, MUTED, true)
    c.drawText(right, contentRight - p.measureText(right), PAGE_H - FOOTER_H + 18f, p)
}

private fun heading(text: String) {
    ensure(64f)
    y += 14f
    val l = text.lowercase()
    val color = when {
        "hotfix" in l -> RED
        "friction" in l || "trend" in l || "drop" in l -> AMBER
        "roadmap" in l || "strategic" in l -> BLUE
        else -> NAVY
    }
    canvas?.drawRoundRect(RectF(MARGIN, y, MARGIN + 5f, y + 20f), 2f, 2f, fill(color))
    paragraph(text, MARGIN + 14f, contentRight - MARGIN - 14f, 13.5f, NAVY, true, lineFactor = 1.35f)
    canvas?.drawRect(MARGIN, y + 1f, contentRight, y + 1.8f, fill(PANEL))
    y += 10f
}

private fun bullet(b: Block.Bullet) {
    val x0 = MARGIN + 6f + b.indent * 14f
    ensure(16f)
    if (b.marker == null) {
        canvas?.drawCircle(x0 + 2.5f, y + 7.5f, 2.2f, fill(ACCENT))
    } else {
        canvas?.drawText(b.marker, x0, y + 10.5f, paint(10.5f, MUTED, true))
    }
    val indentW = if (b.marker == null) 14f else 20f
    paragraph(b.text, x0 + indentW, contentRight - (x0 + indentW), 10.5f, INK, false)
    y += 3f
}

/** Word-wrapped text with bold spans; flows across pages. */
private fun paragraph(raw: String, x0: Float, maxW: Float, size: Float, color: Int, baseBold: Boolean, lineFactor: Float = 1.5f) {
    val regular = paint(size, color, false)
    val bold = paint(size, color, true)
    val lineH = size * lineFactor
    val space = regular.measureText(" ")
    val words = spans(raw).flatMap { (t, b) ->
        t.split(Regex("\\s+")).filter { it.isNotEmpty() }.map { it to (b || baseBold) }
    }
    ensure(lineH)
    var x = x0
    for ((w, isBold) in words) {
        val p = if (isBold) bold else regular
        val ww = p.measureText(w)
        if (x > x0 && x + ww > x0 + maxW) {
            y += lineH; ensure(lineH); x = x0
        }
        canvas?.drawText(w, x, y + size, p)
    }
}

```

```

        x += ww + space
    }
    y += lineH
}
}
}

```

android/app/src/main/java/com/example/clarityai/ui/Theme.kt

44 lines

```

package com.example.clarityai.ui

import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Typography
import androidx.compose.material3.darkColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

val Navy = Color(0xFF0F172A)
val Surface1 = Color(0xFF1E293B)
val Surface2 = Color(0xFF273449)
val Border = Color(0xFF334155)
val Neon = Color(0xFF38BDF8)
val TextPrimary = Color(0xFFE2E8F0)
val TextMuted = Color(0xFF94A3B8)
val Good = Color(0xFF34D399)
val Warn = Color(0xFFFBF24)
val Bad = Color(0xFF871717)

private val ClarityColors = darkColorScheme(
    primary = Neon, onPrimary = Navy,
    secondary = Neon, onSecondary = Navy,
    background = Navy, onBackground = TextPrimary,
    surface = Surface1, onSurface = TextPrimary,
    surfaceVariant = Surface2, onSurfaceVariant = TextMuted,
    surfaceContainer = Surface1, surfaceContainerHigh = Surface2,
    outline = Border, error = Bad,
)

private val ClarityTypography = Typography(
    headlineMedium = TextStyle(fontSize = 28.sp, fontWeight = FontWeight.Bold, letterSpacing = (-0.5).sp),
    titleLarge = TextStyle(fontSize = 20.sp, fontWeight = FontWeight.SemiBold),
    titleMedium = TextStyle(fontSize = 16.sp, fontWeight = FontWeight.SemiBold),
    bodyMedium = TextStyle(fontSize = 14.sp, lineHeight = 21.sp),
    labelMedium = TextStyle(fontSize = 12.sp, fontWeight = FontWeight.Medium, letterSpacing = 0.4.sp),
)

@Composable
fun ClarityTheme(content: @Composable () -> Unit) {
    MaterialTheme(colorScheme = ClarityColors, typography = ClarityTypography, content = content)
}

```

android/app/src/main/java/com/example/clarityai/ui/DashboardViewModel.kt

228 lines

```

package com.example.clarityai.ui

import android.app.Application
import android.content.Context
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.ViewModelScope
import com.android.billingclient.api.Purchase
import com.example.clarityai.billing.BillingManager
import com.example.clarityai.billing.PlanOffer
import com.example.clarityai.data.ApiClient
import com.example.clarityai.data.ApiException
import com.example.clarityai.data.AuditEvent
import com.example.clarityai.data.AuditRequestDto
import com.example.clarityai.data.Entitlements
import com.example.clarityai.data.Kpis
import com.example.clarityai.data.Tier
import com.example.clarityai.data.Timeframe
import com.example.clarityai.pdf.PdfExportUtility
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.Job
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharedFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asSharedFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

```

```

import java.io.File
import java.time.LocalDate

data class PaywallState(val requiredTier: Tier, val reason: String)

data class DashboardUiState(
    val projectId: String = "",
    val timeframe: Timeframe = Timeframe.TODAY,
    val customStart: LocalDate? = null,
    val customEnd: LocalDate? = null,
    val loading: Boolean = false,
    val exporting: Boolean = false,
    val markdown: String = "",
    val kpis: Kpis? = null,
    val notes: List<String> = emptyList(),
    val lockedMessage: String? = null,
    val error: String? = null,
    val entitlements: Entitlements = Entitlements(),
    val paywall: PaywallState? = null,
    val offers: List<PlanOffer> = emptyList(),
    val reportTimeframeLabel: String = "",
)

sealed interface UiEvent {
    data class SharePdf(val file: File) : UiEvent
    data class Message(val text: String) : UiEvent
}

class DashboardViewModel(app: Application) : AndroidViewModel(app) {
    private val prefs = app.getSharedPreferences("clarity_ai", Context.MODE_PRIVATE)
    private val api = ApiClient(app)
    private val projectIdRegex = Regex("[A-Za-z0-9_\\-]{1,128}")

    private val _state = MutableStateFlow(DashboardUiState(projectId = prefs.getString("project_id", "").orEmpty()))
    val state: StateFlow<DashboardUiState> = _state.asStateFlow()

    private val _events = MutableSharedFlow<UiEvent>(extraBufferCapacity = 8)
    val events: SharedFlow<UiEvent> = _events.asSharedFlow()

    private var auditJob: Job? = null

    val billing = BillingManager(
        context = app,
        scope = viewModelScope,
        onPurchase = ::onPurchase,
        onMessage = { _events.tryEmit(UiEvent.Message(it)) },
    )

    init {
        refreshEntitlements()
        viewModelScope.launch { billing.connectAndLoad() }
        viewModelScope.launch { billing.offers.collect { o -> _state.update { it.copy(offers = o) } } }
    }

    fun setProjectId(v: String) = _state.update { it.copy(projectId = v.trim().take(128)) }
    fun setTimeframe(tf: Timeframe) {
        val allowed = tf.name in _state.value.entitlements.features.allowedTimeframes
        if (!allowed) {
            showPaywall(tf.requiredTier, "${tf.label} reports require the ${tf.requiredTier.label()} plan.")
            return
        }
        _state.update { it.copy(timeframe = tf) }
    }
    fun setCustomRange(start: LocalDate, end: LocalDate) =
        _state.update { it.copy(timeframe = Timeframe.CUSTOM, customStart = start, customEnd = end) }

    fun showPaywall(tier: Tier, reason: String) = _state.update { it.copy(paywall = PaywallState(tier, reason)) }
    fun dismissPaywall() = _state.update { it.copy(paywall = null) }
    fun openUpgrade() = showPaywall(if (_state.value.entitlements.tier == Tier.FREE) Tier.PRO else Tier.MAX, "Choose the plan that fits your team.")

    fun refreshEntitlements() {
        viewModelScope.launch {
            runCatching { api.entitlements() }.onSuccess { e -> _state.update { it.copy(entitlements = e) } }
        }
    }

    fun runAudit() {
        val s = _state.value
        if (s.loading) return
        val pid = s.projectId.trim()
        if (!projectIdRegex.matches(pid)) {
            _state.update { it.copy(error = "Enter a valid Clarity project ID (letters, numbers, - and _).") }
            return
        }
        if (s.timeframe.name !in s.entitlements.features.allowedTimeframes) {

```

```

        showPaywall(s.timeframe.requiredTier, "${s.timeframe.label} reports require the ${s.timeframe.requiredTier.label()} 4
plan.")
    }
    return
}
if (s.timeframe == Timeframe.CUSTOM && (s.customStart == null || s.customEnd == null)) {
    _state.update { it.copy(error = "Pick a start and end date for the custom range.") }
    return
}
prefs.edit().putString("project_id", pid).apply()
val dto = AuditRequestDto(
    projectId = pid,
    timeframe = s.timeframe.name,
    startDate = s.customStart?.takeIf { s.timeframe == Timeframe.CUSTOM }?.toString(),
    endDate = s.customEnd?.takeIf { s.timeframe == Timeframe.CUSTOM }?.toString(),
)
auditJob?.cancel()
auditJob = viewModelScope.launch {
    _state.update {
        it.copy(loading = true, markdown = "", kpis = null, error = null, lockedMessage = null,
            notes = emptyList(), reportTimeframeLabel = s.timeframe.label)
    }
    val sb = StringBuilder()
    var lastPush = 0L
    try {
        api.streamAudit(dto).collect { ev ->
            when (ev) {
                is AuditEvent.Meta -> _state.update { st ->
                    st.copy(
                        kpis = ev.kpis, notes = ev.notes,
                        entitlements = st.entitlements.copy(usage = ev.usage ?: st.entitlements.usage),
                    )
                }
                is AuditEvent.Delta -> {
                    sb.append(ev.text)
                    val now = System.currentTimeMillis()
                    if (now - lastPush > 60) { // throttle markdown re-parsing
                        lastPush = now
                        _state.update { it.copy(markdown = sb.toString()) }
                    }
                }
                is AuditEvent.Locked -> _state.update { it.copy(lockedMessage = ev.message) }
                is AuditEvent.Failure -> _state.update { it.copy(error = ev.message) }
                AuditEvent.Done -> Unit
            }
        }
    } catch (e: CancellationException) {
        throw e
    } catch (e: ApiException) {
        handleApiError(e)
    } catch (e: Exception) {
        _state.update { it.copy(error = "Something went wrong. Please try again.") }
    } finally {
        _state.update { it.copy(loading = false, markdown = sb.toString()) }
        refreshEntitlements()
    }
}

private fun handleApiError(e: ApiException) {
    when (e.code) {
        "UPGRADE_REQUIRED", "QUOTA_EXCEEDED", "PROJECT_LIMIT" -> {
            val tier = e.requiredTier
            if (tier != null) showPaywall(tier, e.message)
            else _state.update { it.copy(error = e.message) }
        }
        else -> _state.update { it.copy(error = e.message) }
    }
}

fun exportPdf() {
    val s = _state.value
    if (!s.entitlements.features.pdfExport) {
        showPaywall(Tier.PRO, "PDF export is available on Pro and Max.")
        return
    }
    if (s.markdown.isBlank() || s.loading) {
        _events.tryEmit(UiEvent.Message("Run an audit first, then export the report."))
        return
    }
    viewModelScope.launch {
        _state.update { it.copy(exporting = true) }
        try {
            val file = PdfExportUtility.generate(
                getApplication(), s.markdown,
                PdfExportUtility.ReportMeta(
                    projectId = s.projectId, timeframeLabel = s.reportTimeframeLabel, kpis = s.kpis,

```

```

        notes = s.notes, whiteLabel = s.entitlements.features.whiteLabelPdf,
    ),
)
_events.emit(UiEvent.SharePdf(file))
} catch (e: Exception) {
    _events.emit(UiEvent.Message("Couldn't create the PDF: ${e.message ?: "unknown error"}"))
} finally {
    _state.update { it.copy(exporting = false) }
}
}

private suspend fun onPurchase(p: Purchase): Boolean = try {
    val productId = p.products.firstOrNull() ?: return false
    val ent = api.verifySubscription(p.purchaseToken, productId)
    _state.update { it.copy(entitlements = ent, paywall = null) }
    _events.emit(UiEvent.Message("${ent.tier.label()} plan activated. Thank you!"))
    true
} catch (e: ApiException) {
    _events.emit(UiEvent.Message("Couldn't verify purchase: ${e.message}"))
    false
}

override fun onCleared() {
    billing.close()
    super.onCleared()
}
}

fun Tier.label(): String = name.lowercase().replaceFirstChar { it.uppercase() }

```

android/app/src/main/java/com/example/clarityai/ui/DashboardScreen.kt

393 lines

package com.example.clarityai.ui

```

import android.app.Activity
import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowDropDown
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.PlayArrow
import androidx.compose.material.icons.outlined.AutoAwesome
import androidx.compose.material.icons.outlined.ErrorOutline
import androidx.compose.material.icons.outlined.PictureAsPdf
import androidx.compose.material.icons.outlined.WorkspacePremium
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CenterAlignedTopAppBar
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.DatePickerDialog
import androidx.compose.material3.DateRangePicker
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExtendedFloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.LinearProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField

```

```

import androidx.compose.material3.OutlinedTextFieldDefaults
import androidx.compose.material3.Scaffold
import androidx.compose.material3.SelectableDates
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.material3.rememberDateRangePickerState
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Shape
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardCapitalization
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.example.clarityai.data.Kpis
import com.example.clarityai.data.Tier
import com.example.clarityai.data.Timeframe
import com.example.clarityai.pdf.PdfExportUtility
import com.mikepenz.markdown.m3.Markdown
import com.mikepenz.markdown.m3.markdownColor
import java.time.Instant
import java.time.LocalDate
import java.time.ZoneOffset

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DashboardScreen(vm: DashboardViewModel, activity: Activity) {
    val state by vm.state.collectAsStateWithLifecycle()
    val snackbar = remember { SnackbarHostState() }
    val context = LocalContext.current
    var showDatePicker by remember { mutableStateOf(false) }

    LaunchedEffect(Unit) {
        vm.events.collect { ev ->
            when (ev) {
                is UiEvent.SharePdf -> PdfExportUtility.share(context, ev.file)
                is UiEvent.Message -> snackbar.showSnackbar(ev.text)
            }
        }
    }

    Scaffold(
        containerColor = Navy,
        snackbarHost = { SnackbarHost(snackbar) },
        topBar = {
            CenterAlignedTopAppBar(
                title = {
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Text("Clarity AI", fontWeight = FontWeight.Bold)
                        TierBadge(state.entitlements.tier)
                    }
                },
                navigationIcon = {
                    if (state.entitlements.tier != Tier.MAX) {
                        IconButton(onClick = vm::openUpgrade) { Icon(Icons.Outlined.WorkspacePremium, "Upgrade plan", tint = Neon) }
                    }
                },
                actions = {
                    IconButton(onClick = vm::exportPdf, enabled = !state.exporting) {
                        if (state.exporting) CircularProgressIndicator(Modifier.size(22.dp), strokeWidth = 2.dp, color = Neon)
                        else Box {
                            Icon(Icons.Outlined.PictureAsPdf, "Export PDF report", tint = if (state.entitlements.features.pdfExport) Neon else TextMuted)
                            if (!state.entitlements.features.pdfExport) {
                                Icon(Icons.Filled.Lock, null, tint = Warn, modifier = Modifier.size(11.dp).align(Alignment.BottomEnd))
                            }
                        }
                    }
                },
                colors = TopAppBarDefaults.centerAlignedTopAppBarColors(containerColor = Navy, titleContentColor = TextPrimary),
            )
        }
    )

```

```

    )
  },
  floatingActionButton = {
    ExtendedFloatingActionButton(
      onClick = vm::runAudit,
      containerColor = Neon, contentColor = Navy,
      icon = {
        if (state.loading) CircularProgressIndicator(Modifier.size(20.dp), strokeWidth = 2.dp, color = Navy)
        else Icon(Icons.Filled.PlayArrow, null)
      },
      text = { Text(if (state.loading) "Analysing..." else "Run audit", fontWeight = FontWeight.Bold) },
    )
  },
) { padding ->
  LazyColumn(
    modifier = Modifier.padding(padding),
    contentPadding = PaddingValues(start = 16.dp, end = 16.dp, top = 4.dp, bottom = 96.dp),
    verticalArrangement = Arrangement.spacedBy(16.dp),
  ) {
    item { KpiRow(state.kpis, state.loading) }
    item {
      ControlsCard(
        projectId = state.projectId, onProjectId = vm::setProjectId,
        timeframe = state.timeframe, allowed = state.entitlements.features.allowedTimeframes,
        customLabel = if (state.customStart != null && state.customEnd != null) "${state.customStart} to ${state.customEnd}" else null,
        onTimeframe = { tf ->
          if (tf == Timeframe.CUSTOM && Timeframe.CUSTOM.name in state.entitlements.features.allowedTimeframes) showDatePicker = true
          else vm.setTimeframe(tf)
        },
        usageText = state.entitlements.usage.let { u -> u.limit?.let { "${u.used}/$it audits used this month" } },
      )
    }
    item {
      ReportCard(state, onUpgrade = vm::openUpgrade, onRetry = vm::runAudit)
    }
  }
}

if (showDatePicker) {
  DateRangeDialog(
    onDismiss = { showDatePicker = false },
    onConfirm = { s, e -> vm.setCustomRange(s, e); showDatePicker = false },
  )
}

state.paywall?.let { pw ->
  PaywallSheet(
    currentTier = state.entitlements.tier, paywall = pw, offers = state.offers,
    onSelect = { vm.billing.launch(activity, it) },
    onRestore = vm.billing::restore,
    onDismiss = vm::dismissPaywall,
  )
}
}

@Composable
private fun TierBadge(tier: Tier) {
  val color = when (tier) { Tier.FREE -> TextMuted; Tier.PRO -> Neon; Tier.MAX -> Warn }
  Text(
    tier.label().uppercase(), color = color, fontSize = 10.sp, fontWeight = FontWeight.Bold, letterSpacing = 1.2.sp,
    modifier = Modifier.clip(RoundedCornerShape(50)).background(color.copy(alpha = 0.14f)).padding(horizontal = 8.dp, vertical = 1.dp),
  )
}

// ----- KPI header
@Composable
private fun KpiRow(kpis: Kpis?, loading: Boolean) {
  if (kpis == null) {
    LazyRow(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
      items(4) { if (loading) ShimmerBox(Modifier.size(width = 168.dp, height = 112.dp), RoundedCornerShape(18.dp)) else KpiCard("-", "-", "Run an audit", TextMuted) }
    }
    return
  }
  val healthColor = when { kpis.healthScore >= 80 -> Good; kpis.healthScore >= 60 -> Warn; else -> Bad }
  val cards = listOf(
    Triple("Rage Clicks", "%d".format(kpis.rageClickCount), "${kpis.rageSessionPct}% of sessions") to Bad,
    Triple("Dead-Click Drop-offs", "${kpis.deadClickSessionPct}%", "%d dead clicks".format(kpis.deadClickCount)) to Warn,
    Triple("Session Health", "${kpis.healthScore}", "out of 100") to healthColor,
    Triple("Quick Backs", "${kpis.quickbackSessionPct}%", "of sessions") to Neon,
    Triple("Rapid Scrolls", "${kpis.rapidScrollSessionPct}%", "of sessions") to Neon,
  )
  LazyRow(horizontalArrangement = Arrangement.spacedBy(12.dp)) {

```

```

        items(cards) { (t, color) -> KpiCard(t.first, t.second, t.third, color) }
    }
}

@Composable
private fun KpiCard(title: String, value: String, sub: String, accent: Color) {
    Card(
        modifier = Modifier.size(width = 168.dp, height = 112.dp),
        shape = RoundedCornerShape(18.dp),
        colors = CardDefaults.cardColors(containerColor = Surface1),
        border = BorderStroke(1.dp, Border),
    ) {
        Column(Modifier.padding(14.dp), verticalArrangement = Arrangement.SpaceBetween) {
            Text(title, color = TextMuted, style = MaterialTheme.typography.labelMedium)
            Text(value, color = accent, fontSize = 30.sp, fontWeight = FontWeight.Bold)
            Text(sub, color = TextMuted, style = MaterialTheme.typography.labelMedium)
        }
    }
}

// ----- controls
@Composable
private fun ControlsCard(
    projectId: String, onProjectId: (String) -> Unit,
    timeframe: Timeframe, allowed: List<String>, customLabel: String?,
    onTimeframe: (Timeframe) -> Unit, usageText: String?,
) {
    var expanded by remember { mutableStateOf(false) }
    Card(shape = RoundedCornerShape(18.dp), colors = CardDefaults.cardColors(containerColor = Surface1), border = BorderStroke(1.dp, Border)) {
        Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
            OutlinedTextField(
                value = projectId, onValueChange = onProjectId, singleLine = true,
                label = { Text("Clarity project ID") }, modifier = Modifier.fillMaxWidth(),
                keyboardOptions = KeyboardOptions(capitalization = KeyboardCapitalization.None),
                colors = OutlinedTextFieldDefaults.colors(
                    focusedBorderColor = Neon, unfocusedBorderColor = Border, focusedLabelColor = Neon,
                    unfocusedLabelColor = TextMuted, cursorColor = Neon,
                ),
            ),
            Box {
                OutlinedButton(
                    onClick = { expanded = true }, modifier = Modifier.fillMaxWidth(),
                    border = BorderStroke(1.dp, Border), colors = ButtonDefaults.outlinedButtonColors(contentColor = TextPrimary),
                ),
                Column {
                    Text(if (timeframe == Timeframe.CUSTOM && customLabel != null) customLabel else timeframe.label, Modifier.weight(1f))
                    Icon(Icons.Filled.ArrowDropDown, null)
                }
                DropdownMenu(expanded = expanded, onDismissRequest = { expanded = false }, containerColor = Surface2) {
                    Timeframe.entries.forEach { tf ->
                        val locked = tf.name !in allowed
                        DropdownMenuItem(
                            text = { Text(tf.label + if (locked) " (${"tf.requiredTier.label()})" else "", color = if (locked) TextMuted else TextPrimary) },
                            trailingIcon = { if (locked) Icon(Icons.Filled.Lock, null, tint = Warn, modifier = Modifier.size(16.dp)) },
                            onClick = { expanded = false; onTimeframe(tf) },
                        )
                    }
                }
            }
            usageText?.let { Text(it, color = TextMuted, style = MaterialTheme.typography.labelMedium) }
        }
    }
}

// ----- report
@Composable
private fun ReportCard(state: DashboardUiState, onUpgrade: () -> Unit, onRetry: () -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth(), shape = RoundedCornerShape(20.dp),
        colors = CardDefaults.cardColors(containerColor = Surface1), border = BorderStroke(1.dp, Border),
    ) {
        Column(Modifier.padding(20.dp), verticalArrangement = Arrangement.spacedBy(14.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(Icons.Outlined.AutoAwesome, null, tint = Neon)
                Text("Executive Backlog", Modifier.padding(start = 10.dp), style = MaterialTheme.typography.titleLarge)
            }
            if (state.loading) LinearProgressIndicator(Modifier.fillMaxWidth(), color = Neon, trackColor = Border)
            state.notes.forEach { Text(it, color = Warn, style = MaterialTheme.typography.labelMedium) }

            when {
                state.loading && state.markdown.isBlank() -> ShimmerReport()
                state.error != null && state.markdown.isBlank() -> ErrorBlock(state.error, onRetry)
            }
        }
    }
}

```

```

        state.markdown.isBlank() -> Text(
            "Enter your Clarity project ID, choose a timeframe and run an audit. You'll get a prioritised hotfix list, fr
iction trends and a sprint roadmap.",
            color = TextMuted,
        )
        else -> Markdown(
            content = state.markdown,
            colors = markdownColor(text = TextPrimary, codeBackground = Surface2, dividerColor = Border),
        )
    }
    if (state.error != null && state.markdown.isNotBlank()) Text(state.error, color = Bad)
    state.lockedMessage?.let { LockedBanner(it, onUpgrade) }
}
}

@Composable
private fun LockedBanner(message: String, onUpgrade: () -> Unit) {
    Column(
        Modifier.fillMaxWidth().clip(RoundedCornerShape(14.dp)).background(Surface2).padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(10.dp),
    ) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Icon(Icons.Filled.Lock, null, tint = Warn, modifier = Modifier.size(18.dp))
            Text("Strategic Roadmap locked", Modifier.padding(start = 8.dp), fontWeight = FontWeight.SemiBold)
        }
        Text(message, color = TextMuted, style = MaterialTheme.typography.bodyMedium)
        Button(onClick = onUpgrade, colors = ButtonDefaults.buttonColors(containerColor = Neon, contentColor = Navy)) { Text("Se
e plans", fontWeight = FontWeight.Bold) }
    }
}

@Composable
private fun ErrorBlock(message: String, onRetry: () -> Unit) {
    Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
        Row(verticalAlignment = Alignment.Top) {
            Icon(Icons.Outlined.ErrorOutline, null, tint = Bad)
            Text(message, Modifier.padding(start = 10.dp), color = TextPrimary)
        }
        TextButton(onClick = onRetry) { Text("Try again", color = Neon) }
    }
}

// ----- shimmer
@Composable
fun ShimmerBox(modifier: Modifier = Modifier, shape: Shape = RoundedCornerShape(10.dp)) {
    val transition = rememberInfiniteTransition(label = "shimmer")
    val shift by transition.animateFloat(
        initialValue = -400f, targetValue = 1400f,
        animationSpec = infiniteRepeatable(tween(1300, easing = LinearEasing)), label = "shift",
    )
    val brush = Brush.linearGradient(
        colors = listOf(Border.copy(alpha = 0.35f), Color(0xFF475569).copy(alpha = 0.6f), Border.copy(alpha = 0.35f)),
        start = Offset(shift, 0f), end = Offset(shift + 420f, 140f),
    )
    Box(modifier.clip(shape).background(brush))
}

@Composable
private fun ShimmerReport() {
    Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
        listOf(0.5f, 1f, 0.92f, 0.8f, 0.45f, 1f, 0.88f, 0.6f, 0.5f, 0.95f, 0.7f).forEachIndexed { i, w ->
            val heading = i == 0 || i == 4 || i == 8
            ShimmerBox(Modifier.fillMaxWidth(w).height(if (heading) 22.dp else 13.dp))
            if (heading) Spacer(Modifier.height(2.dp))
        }
    }
}

// ----- date range
@OptIn(ExperimentalMaterial3Api::class)
@Composable
private fun DateRangeDialog(onDismiss: () -> Unit, onConfirm: (LocalDate, LocalDate) -> Unit) {
    val state = rememberDateRangePickerState(
        selectableDates = object : SelectableDates {
            override fun isSelectableDate(utcTimeMillis: Long) = utcTimeMillis <= System.currentTimeMillis() + 86_400_000L
        },
    )
    val start = state.selectedStartDateMillis
    val end = state.selectedEndDateMillis
    DatePickerDialog(
        onDismissRequest = onDismiss,
        confirmButton = {
            TextButton(
                enabled = start != null && end != null,
                onClick = {

```

```

        val s = Instant.ofEpochMilli(start!!).atZone(ZoneOffset.UTC).toLocalDate()
        val e = Instant.ofEpochMilli(end!!).atZone(ZoneOffset.UTC).toLocalDate()
        onConfirm(s, e)
    },
    ) { Text("Apply") }
},
dismissButton = { TextButton(onClick = onDismiss) { Text("Cancel") } },
) {
    DateRangePicker(state = state, modifier = Modifier.height(520.dp))
}
}
}

```

android/app/src/main/java/com/example/clarityai/ui/PaywallSheet.kt

124 lines

```

package com.example.clarityai.ui

import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.ModalBottomSheet
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.rememberModalBottomSheetState
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.clarityai.billing.PlanOffer
import com.example.clarityai.data.Tier

private data class PlanInfo(val tier: Tier, val tagline: String, val features: List<String>)

// Keep in sync with backend services/tiers.py
private val PLANS = listOf(
    PlanInfo(Tier.FREE, "Try it on one project", listOf(
        "3 audits / month", "Today & last 3 days", "Top 3 friction pages",
        "Hotfixes + friction trends (roadmap locked)", "1 project")),
    PlanInfo(Tier.PRO, "For product teams", listOf(
        "60 audits / month", "Up to last 7 days", "Top 10 friction pages",
        "Full strategic sprint roadmap", "Executive PDF export & share", "5 projects")),
    PlanInfo(Tier.MAX, "For agencies & orgs", listOf(
        "600 audits / month", "Last 30 days + custom ranges (90 days)", "Device-level breakdown",
        "White-label PDF reports", "Everything in Pro", "25 projects")),
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PaywallSheet(
    currentTier: Tier,
    paywall: PaywallState,
    offers: List<PlanOffer>,
    onSelect: (PlanOffer) -> Unit,
    onRestore: () -> Unit,
    onDismiss: () -> Unit,
) {
    ModalBottomSheet(
        onDismissRequest = onDismiss,
        sheetState = rememberModalBottomSheetState(skipPartiallyExpanded = true),
        containerColor = Navy,
        contentColor = TextPrimary,
    ) {
        Column(
            Modifier.verticalScroll(rememberScrollState()).padding(horizontal = 20.dp).navigationBarsPadding(),
            verticalArrangement = Arrangement.spacedBy(14.dp),
        ) {

```

```

Text("Upgrade your plan", style = androidx.compose.material3.MaterialTheme.typography.titleLarge)
Text(paywall.reason, color = TextMuted, style = androidx.compose.material3.MaterialTheme.typography.bodyMedium)

PLANS.forEach { plan ->
    val offer = offers.firstOrNull { it.tier == plan.tier }
    val isCurrent = plan.tier == currentTier
    val highlight = plan.tier == paywall.requiredTier
    Card(
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = Surface1),
        border = BorderStroke(if (highlight) 1.5.dp else 1.dp, if (highlight) Neon else Border),
    ) {
        Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(6.dp)) {
            Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween, verticalAlignment = Alignment.CenterVertically) {
                Column {
                    Text(plan.tier.label(), fontWeight = FontWeight.Bold, style = androidx.compose.material3.MaterialTheme.typography.titleMedium)
                    Text(plan.tagline, color = TextMuted, style = androidx.compose.material3.MaterialTheme.typography.labelMedium)
                }
                Text(
                    if (plan.tier == Tier.FREE) "Free" else offer?.let { "${it.price}/mo" } ?: "-",
                    color = Neon, fontWeight = FontWeight.Bold,
                )
            }
            Spacer(Modifier.height(4.dp))
            plan.features.forEach { f ->
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Filled.CheckCircle, null, tint = Good, modifier = Modifier.size(16.dp))
                    Text(f, Modifier.padding(start = 8.dp), style = androidx.compose.material3.MaterialTheme.typography.bodyMedium)
                }
            }
            Spacer(Modifier.height(6.dp))
            when {
                isCurrent -> OutlinedButton(onClick = {}, enabled = false, modifier = Modifier.fillMaxWidth()) { Text("Current plan") }
                plan.tier.ordinal < currentTier.ordinal -> Unit
                offer == null -> OutlinedButton(onClick = {}, enabled = false, modifier = Modifier.fillMaxWidth()) { Text("Unavailable") }
                else -> Button(
                    onClick = { onSelect(offer) },
                    modifier = Modifier.fillMaxWidth(),
                    colors = ButtonDefaults.buttonColors(containerColor = Neon, contentColor = Navy),
                ) { Text("Choose ${plan.tier.label()}", fontWeight = FontWeight.Bold) }
            }
        }
    }
}
TextButton(onClick = onRestore, modifier = Modifier.align(Alignment.CenterHorizontally)) { Text("Restore purchases", color = Neon) }
Text(
    "Subscriptions renew monthly through Google Play and can be cancelled anytime in the Play Store.",
    color = TextMuted, style = androidx.compose.material3.MaterialTheme.typography.labelMedium,
)
Spacer(Modifier.height(24.dp))
}
}
}

```